
NutriTrack

Manual de inicio del Front

.NET MAUI · MVVM · Conexión con la API · Fase 0

Guía paso a paso, construida a partir del desarrollo real de la estructura base del front de NutriTrack: qué se hizo, por qué se hizo así, cómo se comunica cada archivo, cómo habla el front con el back, qué errores aparecieron y cómo se resolvieron. Pensada como punto de partida para todo el equipo.

Proyecto	NutriTrack: sistema de gestión ganadera
Institución	Instituto Tecnológico El Molino · Tecnicatura Superior en Desarrollo de Software
Unidad curricular	Práctica Profesionalizante II · Docente: Kappes Nicolás
Equipo	Lamerata Daniela, Stauber Tomás, Tapia Gala
Rama de trabajo	chore/maui-estructura-base
Fecha	Octubre de 2026

Índice

1. Cómo usar este manual	5
Convenciones del manual	5
2. El proyecto NutriTrack	6
2.1 Qué es y para quién	6
2.2 Stack tecnológico	6
2.3 Cómo está armado el back	6
2.4 Estado inicial del front	7
3. Análisis inicial: qué encontramos antes de programar	8
3.1 Endpoints que el front va a necesitar y no existían	8
3.2 Bloqueo 1: no existía el endpoint de login	8
3.3 Bloqueo 2: los usuarios de prueba no pueden loguearse	9
3.4 Bloqueo 3: la API dentro de Docker no tiene la clave del JWT	9
3.5 El PR del login (hecho por un compañero)	10
4. Plan de desarrollo del front por fases	11
4.1 Por qué la Fase 0 va en su propio PR	11
4.2 La Fase 0 hace la parte invisible del login	12
5. Conceptos fundamentales	13
5.1 El problema que resuelve MVVM	13
5.2 MVVM: separar en capas	13
5.3 Binding: el pegamento	14
5.4 Code-behind: los archivos que vienen de a dos	14
5.5 Inyección de dependencias (DI)	15
5.6 Interfaces e inversión de dependencias	15
5.7 HTTP, JSON y códigos de estado	16
5.8 JWT y el encabezado Bearer	16
5.9 Middleware (back) y DelegatingHandler (front)	16
5.10 Patrón Result: errores como resultados	16
5.11 async/await, hilos y el hilo principal	17
5.12 Recursos y estilos: Colors.xaml y Styles.xaml	17
5.13 Shell y navegación	17
5.14 Compilación condicional: #if	18
6. Git y trabajo en equipo	19
6.1 Los dos lugares de git	19
6.2 Leer el git status	19
6.3 Convención de ramas y commits	19

6.4 El flujo que se usó	20
6.5 Comandos de rescate	20
6.6 Abrir un Pull Request	20
7. Fase 0 paso a paso	22
Paso 1: verificar que MAUI esté instalado	22
Paso 2: crear la rama	23
Paso 3: correr la plantilla tal como está	23
Paso 4: instalar los paquetes NuGet	24
Paso 5: dejar solo Windows y Android	24
Paso 6: MVVM y carpetas	25
Paso 7: colores del prototipo y tema claro	25
Paso 8: primer commit	28
Paso 9: conectar el front con la API	28
Paso 10: quitar la prueba y guardar	30
Paso 11: la sesión del usuario	30
Paso 12: el handler del token	33
Paso 13: manejo de errores de la API	35
Paso 14: el BaseViewModel	39
Paso 15: navegación con Shell y pantallas provisionarias	40
Resumen de commits de la rama	45
8. Estructura final: qué va en cada archivo	46
8.1 Tabla de responsabilidades	46
8.2 Convención de nombres	47
9. Cómo se comunican los archivos	48
9.1 Secuencia de arranque de la app	48
9.2 Recorrido de un pedido (ejemplo: Ingresar)	48
9.3 Recorrido cuando el token vence	49
9.4 Mapa de dependencias	49
9.5 Ciclos de vida registrados	50
10. Comunicación front ↔ back	51
10.1 Contrato del login	51
10.2 Reglas que front y back tienen que compartir	51
10.3 Cómo levantar todo para desarrollar	51
10.4 Probar la API sin el front	52
11. Cómo agregar una pantalla nueva	53
Paso A: el modelo (DTO)	53
Paso B: el servicio	53

Paso C: el ViewModel	54
Paso D: la View	54
Paso E: registrar y navegar	56
Checklist	56
12. Catálogo de errores y cómo resolverlos	57
12.0 Reglas generales para cualquier error	57
12.1 a 12.4: git	57
12.5 a 12.7: ejecutar y compilar	58
12.8 a 12.9: colores y recursos	59
12.10 a 12.12: compilación del proyecto MAUI	60
12.13 a 12.14: XAML y orden de ejecución	60
12.15 Advertencias que se pueden ignorar	61
12.16 Dudas de Visual Studio	61
13. Preguntas que surgieron en el proceso	62
14. Decisiones de diseño (para la tesis)	64
Frases listas para el documento	65
15. Pendientes y próximos pasos	66
15.1 Pendientes del equipo	66
15.2 Pendiente de probar (depende de la Fase 1)	66
15.3 Fase 1: el login real	66
16. Comandos y atajos útiles	68
Git	68
.NET y la API	68
Visual Studio	68
17. Glosario	70
Apéndice A. Código final de referencia	71
Apéndice B. Descripción del PR de la Fase 0	85

1. Cómo usar este manual

Este manual documenta la **Fase 0** del front de NutriTrack: la estructura base sobre la que se van a construir todas las pantallas (login, animales, rodeos, planes alimenticios, eventos sanitarios, reportes y usuarios). No es una referencia teórica de MAUI: es el registro de cómo se armó **este** proyecto, con **este** stack, resolviendo **estos** problemas.

Está pensado para tres usos:

- **Aprender desde cero.** Si nunca usaste MAUI, lee los capítulos en orden. Cada concepto se explica antes de usarse, siempre con el "por qué" y no solo el "cómo".
- **Consultar mientras programás.** Los capítulos 8 a 11 (qué va en cada archivo, cómo se comunican, cómo habla el front con el back y cómo agregar una pantalla) son la guía diaria del equipo.
- **Resolver errores.** El capítulo 12 es un catálogo de todos los errores que aparecieron durante la Fase 0, con su síntoma, su causa y su solución.

Convenciones del manual

Elemento	Significado
Texto así	Nombres de archivos, clases, métodos, comandos o valores literales.
Bloques grises con borde verde	Código o comandos. Si dice "reemplazá todo", se selecciona todo el archivo con Ctrl + A y se pega encima.
Recuadro verde "Para la tesis"	Frases y justificaciones listas para usar en la documentación del trabajo.
Recuadro naranja "Atención"	Algo que, si se pasa por alto, genera un error.
Recuadro violeta "Tip"	Atajos o hábitos que ahorran tiempo.

La idea central

Cada decisión de este manual tiene una razón. Si algo parece complicado de más, buscá el párrafo que explica el "por qué": casi siempre es para evitar un problema concreto que aparecería más adelante (código repetido, errores difíciles de encontrar, conflictos entre compañeros o fallas de seguridad).

2. El proyecto NutriTrack

2.1 Qué es y para quién

NutriTrack es un sistema de gestión ganadera desarrollado junto a una empresa agropecuaria de Urdinarrain, Entre Ríos, que hoy gestiona sus registros en papel. Reemplaza ese soporte físico por registros digitales: identificación individual de cada animal, rodeos, control de peso, planes alimenticios, eventos sanitarios, alertas automáticas y reportes.

Tiene tres tipos de usuario con permisos diferenciados: **Administrador**, **Encargado de campo** y **Asesor técnico**. Según el documento de requerimientos, la app debe funcionar en **PC y dispositivos Android** (RNF2), es **solo online** (sección 1.3: no hay modo offline) y está optimizada para hasta 5000 animales.

2.2 Stack tecnológico

Capa	Tecnología	Proyecto en la solución
Front (app)	.NET MAUI con patrón MVVM	NutriTrack.MAUI
API	ASP.NET Core Web API (.NET 10)	NutriTrack.API
Dominio	Entidades y enums	NutriTrack.Core
Acceso a datos	Entity Framework Core + Repositories	NutriTrack.Infraestructure
Base de datos	PostgreSQL en Docker	docker-compose.yml, db/scripts
IDE	Visual Studio 2026 (versión 18.10)	Solución NutriTrack.slnx
Repositorio	GitHub	github.com/tomasstauber/NutriTrack

2.3 Cómo está armado el back

El back sigue un patrón **Controller → Repository**, sin capa de servicios:

- Los **Controllers** (en `NutriTrack.API/Controllers`) reciben los pedidos HTTP, validan las reglas de cada caso de uso (las R1, R2... de las especificaciones) y devuelven la respuesta. Cuando algo está mal devuelven, por ejemplo, `BadRequest("La caravana ya existe")`.
- Los **Repositories** (en `NutriTrack.Infraestructure/Repositories`) hacen las consultas a la base con Entity Framework.
- Los **DTOs** (en `NutriTrack.API/DTOs`) son las clases que viajan como JSON entre la API y quien la consume.
- El esquema de la base lo arma el SQL que levanta Docker; no se usan Migrations.
- La API serializa los **enums como texto** (`JsonStringEnumConverter` en `Program.cs`), por ejemplo `"Administrador"` en vez de `2`.

Al comenzar el front estaban implementados en la API los casos de uso CU1 a CU6, CU8 a CU14, CU17, CU18 y el CRUD de usuarios (CU21 a CU24). El CU19 estaba en una rama sin mergear y faltaban el CU20 y las alertas (CU15 y CU16), que se dejaron para el final porque necesitan parte del front.

2.4 Estado inicial del front

El proyecto `NutriTrack.MAUI` era exactamente la plantilla de Visual Studio: una `MainPage` con el robot `dotnet_bot`, el texto "Hello, World!", un botón "Click me" que contaba clics, colores violetas por defecto y ningún paquete adicional. Es decir: el front se empezó desde cero.

3. Análisis inicial: qué encontramos antes de programar

Antes de escribir una línea del front se revisó el repositorio completo (el código real en GitHub, no solo la documentación). Este paso evitó varios problemas que hubieran aparecido a mitad del trabajo.

Pensamiento de ingeniería

Antes de construir sobre algo, verifiqué cómo está realmente. La documentación dice lo que se planeó; el código dice lo que existe.

3.1 Endpoints que el front va a necesitar y no existían

Falta en la API	Quién lo necesita
Endpoint de login (nada generaba tokens)	El login y, por consiguiente, todo lo demás
GET listado de animales	"Ver Animales" del panel principal
GET listado de rodeos	Pantalla Rodeos y los selectores de CU8, CU11, CU13 y reportes
GET listado de planes alimenticios	Selector de planes del CU11
Resumen del panel (total de animales, activos, próximos eventos)	Panel principal

Acuerdo del equipo: **cada uno agrega el endpoint que necesite su pantalla** en el momento en que la construye.

3.2 Bloqueo 1: no existía el endpoint de login

En `Program.cs` estaba configurada la **validación** de tokens JWT (`AddJwtBearer`, `UseAuthentication`), pero ningún endpoint los **generaba**. Configurar la validación sin la generación es como tener un portero que revisa entradas, pero ninguna boletería que las venda.

Esto se resolvió con el PR de un compañero (ver 3.5). Mientras tanto se planteó una estrategia para no quedar bloqueados: programar contra una **interfaz** `IAuthService` con una implementación falsa (`AuthServiceFalso`) que devolviera un usuario de prueba, y reemplazarla después por la real cambiando una sola línea en `MauiProgram.cs`. Esto es **inversión de dependencias** (ver capítulo 5). Finalmente no hizo falta, porque el login real llegó antes de empezar la Fase 1.

3.3 Bloqueo 2: los usuarios de prueba no pueden loguearse

Aunque exista el endpoint, con la base de datos tal como estaba nadie podía entrar:

1. **Contraseñas falsas.** Los hashes del seed (`$2b$10$hashadmin123`) son texto de relleno con apariencia de bcrypt. No son hashes Argon2 reales, así que la verificación siempre falla.
2. **Roles que no coinciden con el enum.** En la base están guardados como `'administrador'` o `'asesor técnico'`, pero EF usa `HasConversion<string>()` y espera exactamente los nombres del enum: `Administrador`, `AsesorTecnico`, `EncargadoDeCampo`. Leer esos usuarios lanza una excepción.
3. **Falta nombre_usuario en el DML.** El DDL lo define como `NOT NULL UNIQUE`, así que el insert falla.
4. **backup0k.sql tiene el esquema viejo.** Es la base que levanta Docker y su tabla `usuario` no tiene `nombre_usuario` ni `activo`.

El problema del huevo y la gallina

Con el login real activo, **crear usuarios también exige token** (POST `api/Usuario` queda protegido). En una base nueva nadie puede entrar para crear el primer usuario: es el problema del huevo y la gallina.

La solución típica es que el SQL de la base incluya **un administrador inicial con un hash Argon2 real**. Queda como pendiente del equipo (ver capítulo 15).

3.4 Bloqueo 3: la API dentro de Docker no tiene la clave del JWT

Esta fue una de las preguntas del proceso: **¿por qué la clave tiene que ir en el docker-compose?**

Dónde vive la clave

Al ejecutar `dotnet user-secrets set "Jwt:Key" ...` la clave se guarda en un archivo del usuario de Windows (`%APPDATA%\Microsoft\UserSecrets\<id>\secrets.json`), **fuera** de la carpeta del proyecto. Es a propósito: así nunca se sube a GitHub. Cuando la API corre desde Visual Studio, .NET encuentra ese archivo y carga la clave.

Qué pasa dentro de Docker

El `docker-compose.yml` levanta la base (db) y también la API (api). La API se construye con el `Dockerfile`, que copia solo el código del proyecto. `secrets.json` no está en la carpeta del proyecto, así que no viaja. Dentro del contenedor la clave vale `null` y la autenticación falla en cada pedido.

Diagrama



La solución

ASP.NET lee configuración de varias fuentes, entre ellas las **variables de entorno**. Una variable `Jwt__Key` (con doble guion bajo) se traduce a `Jwt:Key`. Es el mismo mecanismo que el compose ya usa para la conexión a la base:

`docker-compose.yml` (servicio api)

```
environment:
  ConnectionStrings__DefaultConnection: "Host=db;..." # ya existía
  Jwt__Key: ${JWT_KEY}                                # lo que falta
```

Se escribe `${JWT_KEY}` y no la clave en sí porque el compose se sube a GitHub. Docker toma el valor del archivo `.env` de la raíz, que ya está en el `.gitignore` (y existe un `.env.example` como plantilla).

Cómo trabajás	¿Necesitás la clave en el compose?
API desde Visual Studio o terminal + solo la base en Docker	No: alcanzan los user secrets
Todo con <code>docker compose up --build</code>	Sí: si no, la API no autentica

Para desarrollar el front se usa la primera forma, así que este bloqueo no impide avanzar.

3.5 El PR del login (hecho por un compañero)

Durante la Fase 0 un compañero subió el login. Resumen de lo que agregó y de lo que significa para el front:

Qué se hizo en el back	Qué significa para el front
<code>POST /api/login</code> valida usuario y contraseña (Argon2id) y devuelve un JWT firmado (HMAC-SHA256) que expira a las 8 horas (UTC).	Es el único endpoint abierto. El front le manda <code>{ nombreUsuario, contrasenia }</code> .
Respuesta: <code>Token, Expiracion, NombreUsuario, Rol</code> .	Es lo que guarda el <code>SesionService</code> . No trae el id ni el nombre completo.
401 con mensaje genérico ante cualquier credencial inválida.	Evita la <i>enumeración de usuarios</i> . El front muestra ese mensaje tal cual.
Fallback policy : todos los endpoints exigen estar autenticado.	Cada pedido del front tiene que llevar <code>Authorization: Bearer <token></code> . Por eso existe el <code>AuthHeaderHandler</code> .
Configuración <code>Jwt:Issuer = NutriTrack.API</code> , <code>Jwt:Audience = NutriTrack.Clientes</code> , y <code>Jwt:Key</code> en user secrets.	Cada dev debe agregar Issuer y Audience a su <code>appsettings.json</code> y la clave a sus user secrets, o la API no arranca (falla con un mensaje claro).
Fuera de alcance: restricción por rol (<code>[Authorize(Roles = ...)]</code>).	El front oculta botones según el rol, pero eso no es seguridad : la seguridad real es del back.

Para la tesis

"Ocultar opciones en la interfaz según el rol mejora la experiencia de usuario, pero no constituye un mecanismo de seguridad: cualquier cliente puede invocar la API directamente. El control de acceso se aplica en el servidor mediante autorización basada en roles."

4. Plan de desarrollo del front por fases

El front se dividió en fases ordenadas por **dependencia**: cada fase usa lo que construyó la anterior.

Fase	Contenido	Casos de uso	Depende de
0. Base	Paquetes, carpetas, conexión con la API, sesión, token, errores, BaseViewModel, estilos, navegación	—	Nada
1. Login	Pantalla de login, LoginViewModel, AuthService, redirección por rol, cerrar sesión	RF7	Fase 0 + endpoint de login
2. Panel principal	Tarjetas de resumen, próximos eventos, accesos directos	—	Fase 1 + endpoint de resumen
3. Animales	Listado, ficha, alta, edición, desactivar y reactivar, registrar peso	CU1-5, CU14	Fase 1
4. Rodeos	Crear, transferir, eliminar	CU6, 8, 9	Fase 1
5. Alimentación	Ingredientes, crear, editar y asignar planes	CU10-12	Fase 4
6. Sanidad	Evento sanitario múltiple, medicamentos	CU13	Fases 3 y 4
7. Reportes	Cuatro reportes + descarga de PDF	CU17-20	Fase 1
8. Usuarios	Listado, crear, editar, eliminar (solo administrador)	CU21-24	Fase 1
9. Alertas	Back de CU15/16 + campana del panel	CU15-16	Todo lo anterior

4.1 Por qué la Fase 0 va en su propio PR

La Fase 0 es la base que van a usar **todas** las pantallas. Si cada integrante arrancara su pantalla por su cuenta, aparecerían tres `HttpClient` configurados distinto, tres formas de manejar errores y conflictos constantes en `MauiProgram.cs`.

- **Habilita el trabajo en paralelo.** Una vez mergeada, cada uno hace `git pull` y construye su pantalla sobre la misma base.
- **Es más fácil de revisar.** Los compañeros revisan "la estructura" sin mezclarla con "el login".
- **Desacopla la revisión.** Si al login le piden cambios, la base ya está mergeada y nadie espera.
- **El historial queda claro** para la tesis: primero la arquitectura, después cada funcionalidad.

Se discutió subir el PR antes de terminar la Fase 0 (solo con la configuración). Se decidió terminarla primero, porque sin el handler del token, el manejo de errores, el `BaseViewModel` y la navegación, los compañeros no podrían hacer pantallas que funcionen contra la API (con el login activo todo devuelve 401 sin token).

4.2 La Fase 0 hace la parte invisible del login

La Fase 0 no tiene pantalla de login, pero construye casi todo lo que el login necesita:

El login necesita...	¿Está en la Fase 0?
Conectarse a la API	Sí: <code>HttpClient</code> + <code>ApiConfig</code>
Guardar el token y el rol después de entrar	Sí: <code>SesionService</code>
Recordar la sesión al reabrir la app	Sí: <code>RestaurarSesionAsync</code> + <code>AppShell</code>
Mostrar "usuario o contraseña incorrectos" o "sin conexión"	Sí: <code>ApiServiceBase</code>
Mostrar "cargando" y evitar el doble clic	Sí: <code>BaseViewModel</code>
Ir al panel al entrar y volver al login si el token vence	Sí: <code>Shell</code> + <code>AuthHeaderHandler</code>
La pantalla con el diseño del prototipo	No: Fase 1
<code>LoginViewModel</code> (qué pasa al tocar "Ingresar")	No: Fase 1
<code>AuthService</code> (el <code>POST api/login</code>)	No: Fase 1

5. Conceptos fundamentales

Este capítulo explica las ideas sobre las que se apoya todo el front. Conviene leerlo antes de mirar el código: cada archivo de la Fase 0 aplica uno o más de estos conceptos.

5.1 El problema que resuelve MVVM

La plantilla de MAUI trae este código en `MainPage.xaml.cs`:

MainPage.xaml.cs (plantilla)

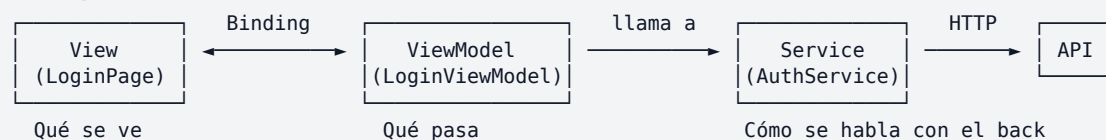
```
private void OnCounterClicked(object? sender, EventArgs e)
{
    count++;
    CounterBtn.Text = $"Clicked {count} times";
}
```

El código **toca el botón directamente** (`CounterBtn.Text = ...`): la lógica ("sumar 1") y la pantalla ("cambiar el texto del botón") están mezcladas. En un contador no importa, pero en el login hay que validar campos, llamar a la API, guardar el token, mostrar errores, deshabilitar el botón mientras carga y navegar. Si todo eso está pegado a los controles, el archivo crece sin orden, la lógica no se puede probar sin abrir la ventana y cambiar el diseño rompe la lógica.

5.2 MVVM: separar en capas

MVVM (Model-View-ViewModel) aplica la misma idea que el back con Controller → Repository: **cada pieza tiene una sola responsabilidad**.

Las capas del front



Capa	Responsabilidad	Qué NO hace	Parecido en el back
View (.xaml)	Dibujar la pantalla: campos, botones, colores	No tiene lógica	La "cara" (como Scalar)
ViewModel	Guardar el estado de la pantalla (lo escrito, si está cargando, el error) y reaccionar a las acciones	No sabe qué controles existen	El Controller
Model	Clases de datos que van y vienen de la API	Nada: solo datos	Los DTOs
Service	Hablar con la API por HTTP	No sabe nada de pantallas	El Repository (pero habla con la API, no con la base)

5.3 Binding: el pegamento

El **Binding** (enlace de datos) conecta la View con el ViewModel. En el XAML se indica "este texto mostralo desde tal propiedad", y cuando la propiedad cambia, la pantalla se actualiza sola. El mismo contador en MVVM, con `CommunityToolkit.Mvvm`:

ViewModel

```
public partial class ContadorViewModel : ObservableObject
{
    [ObservableProperty]
    public partial int Contador { get; set; }

    [RelayCommand]
    private void Incrementar() => Contador++;
}
```

View (XAML)

```
<Button Text="{Binding Contador}" Command="{Binding IncrementarCommand}" />
```

- `ObservableObject`: permite que el ViewModel avise "¡cambié!".
- `[ObservableProperty]`: convierte la propiedad en una que avisa sola cada vez que cambia. Sin el toolkit serían unas 10 líneas por propiedad.
- `[RelayCommand]`: convierte el método en un comando (`IncrementarCommand`) que un botón puede ejecutar.
- `partial`: el toolkit **genera el resto del código** al compilar. Por eso la clase y las propiedades llevan esa palabra.

El ViewModel **nunca menciona el botón**. Solo dice "tengo un número y sé incrementarlo"; la View decide cómo mostrarlo.

Para la tesis

"Se adoptó el patrón MVVM para separar la lógica de presentación de la interfaz gráfica, favoreciendo el bajo acoplamiento y la mantenibilidad (RNF5): la interfaz puede modificarse sin alterar la lógica, y la lógica puede probarse sin la interfaz."

5.4 Code-behind: los archivos que vienen de a dos

Cada pantalla tiene dos archivos agrupados en el Explorador de soluciones (se ven con la flechita a la izquierda del `.xaml`, o con F7):

Archivo	Qué tiene
LoginPage.xaml	El diseño: qué controles hay y dónde.
LoginPage.xaml.cs	El código detrás (<i>code-behind</i>): lo que pasa cuando tocás algo.

Son dos mitades de la **misma clase**: por eso el `.cs` dice `partial class`. El método `InitializeComponent()` lo genera MAUI a partir del XAML. En NutriTrack el code-behind debe quedar casi vacío: la lógica va al ViewModel.

5.5 Inyección de dependencias (DI)

Es el mismo mecanismo que se usa en la API con `builder.Services...`. En el front se configura en `MauiParam.cs`. En lugar de que cada clase cree lo que necesita con `new`, lo **pide en su constructor** y el contenedor de DI se lo entrega ya armado.

```
public AppShell(ISessionService sesionService) // lo pide
{
    _sesionService = sesionService;           // DI se lo da
}
```

Ciclos de vida (lifetimes)

Lifetime	Qué significa	Dónde se usa en NutriTrack
Singleton	Una sola instancia para toda la app	<code>SesionService</code> (tiene que haber UNA sesión)
Transient	Una instancia nueva cada vez que se pide	Pantallas, ViewModels y el <code>AuthHeaderHandler</code>
Scoped	Una instancia por "alcance" (en la API, por pedido HTTP)	Se usa en la API (repositorios). En el front casi no aplica

Atención

Si el `SesionService` fuera Transient, el login guardaría la sesión en una instancia y el panel consultaría otra vacía. El lifetime no es un detalle: define si el estado se comparte o no.

Service locator: lo que NO hay que hacer

En la prueba de conexión (Paso 9) se usó

`IPlatformApplication.Current!.Services.GetRequiredService<...>()` para buscar un servicio "a mano". Eso se llama *service locator* y es mala práctica, porque esconde de qué depende la clase. Solo se usó en una prueba descartable. En el código real los servicios se reciben **por constructor**.

5.6 Interfaces e inversión de dependencias

Una **interfaz** es un contrato: dice qué se puede hacer, no cómo. Las clases dependen del contrato (`ISesionService`) y no de la implementación concreta (`SesionService`).

Cambiar la implementación sin tocar las pantallas

```
builder.Services.AddSingleton<IAuthService, AuthServiceFalso>(); // mientras no hay back
builder.Services.AddSingleton<IAuthService, AuthService>();      // cuando el back existe
```

La pantalla y el ViewModel no cambian: solo cambia una línea de registro. Es la misma idea que se usa en los tests con *mocks*.

5.7 HTTP, JSON y códigos de estado

El front y el back se comunican con **pedidos HTTP** (GET, POST, PUT, DELETE) que llevan datos en **JSON**. Cada respuesta trae un **código de estado** que le dice a la app qué hacer:

Código	Significado	Qué hace el front
200 OK	Salió bien	Usa los datos
400 Bad Request	Datos inválidos o regla de negocio incumplida	Muestra el mensaje del back
401 Unauthorized	Sin token, token vencido o credenciales inválidas	En el login: "usuario o contraseña incorrectos". En el resto: vuelve al login
403 Forbidden	Autenticado pero sin permiso	"No tenés permisos"
404 Not Found	No existe lo pedido	Muestra el mensaje del back
500 Internal Server Error	Falló el servidor (por ejemplo, base caída)	Mensaje genérico, nunca el detalle
(sin respuesta)	No se pudo conectar o timeout	"No se pudo conectar con el servidor"

5.8 JWT y el encabezado Bearer

El **token JWT** es la "llave" que devuelve el login. La API la revisa en cada pedido. Viaja en un encabezado HTTP:

```
GET api/Ingrediente
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Es lo mismo que se hace en Scalar al pegar el token en **Authentication → Bearer**. Contiene *claims* (id, nombre y rol del usuario), está firmado con **Jwt:Key** y vence a las 8 horas.

5.9 Middleware (back) y DelegatingHandler (front)

En la API, cada pedido que **llega** pasa por *middlewares* (`app.UseAuthentication()`) antes del controller. En el front, cada pedido que **sale** puede pasar por un **DelegatingHandler** antes de irse por la red. Uno pone el sello; el otro lo controla.

```
Service → AuthHeaderHandler → red → UseAuthentication → Controller
          (le pega el token)          (revisa el token)
```

5.10 Patrón Result: errores como resultados

Opción	Pro	Contra
Lanzar excepciones	Es lo clásico	Si alguien olvida el try/catch, la app se cierra. Una contraseña incorrecta no es "excepcional"
Devolver un resultado (<code>ResultadoApi<T></code>)	Imposible olvidarse: el resultado siempre trae la respuesta	Un poco más de código al principio

Los errores esperables no son excepciones, son resultados posibles. Que el usuario escriba mal la contraseña o que no haya internet en el campo es parte del funcionamiento normal.

5.11 async/await, hilos y el hilo principal

- Los pedidos HTTP son **asíncronos** (`async Task + await`): mientras se espera la respuesta, la pantalla no se congela.
- `async void` solo se permite en *event handlers* (como `OnEntrarClicked`). En el resto del código siempre `async Task`.
- Las respuestas pueden llegar en un hilo secundario, pero en MAUI **solo el hilo principal puede tocar la interfaz** (mostrar alertas, navegar). Para eso existe `MainThread.BeginInvokeOnMainThread(...)`. Sin eso la app puede crashear de forma aleatoria.

5.12 Recursos y estilos: Colors.xaml y Styles.xaml

Archivo	Qué tiene	Analogía
Resources/Styles/Colors.xaml	La paleta: cada color con un nombre (<code>Primary...</code>)	La caja de pinturas
Resources/Styles/Styles.xaml	Las reglas: "todos los botones usan Primary de fondo"	Las instrucciones de qué pintar

`Styles.xaml` nunca dice "verde": dice `{StaticResource Primary}`. Si cambia el valor de `Primary`, cambian todos los botones a la vez. Es el mismo principio que las constantes en el código.

`AppThemeBinding` elige colores según el tema del sistema: `{AppThemeBinding Light=..., Dark=...}`. Por eso la app se veía oscura en un Windows en modo oscuro, hasta que se fijó el tema claro.

Atención

Los `StaticResource` se buscan por nombre (un texto). El compilador **no verifica** que existan: si falta uno, la app compila y falla recién al abrirse (`XamlParseException`). Además, el recurso tiene que estar cargado **antes** de que alguien lo use.

5.13 Shell y navegación

`Shell` es el esqueleto de navegación de MAUI (`AppShell.xaml`). Cada pantalla tiene una **ruta** y se navega con `GoToAsync("ruta")`.

Tipo	Cómo se escribe	Qué hace	Se usa para
Absoluta	<code>GoToAsync("//panel")</code>	Reemplaza todo y borra el historial: no hay flecha de volver	Solo entrar y salir de la sesión (Login → Panel, cualquier pantalla → Login)
Relativa	<code>GoToAsync("animales")</code>	Apila la pantalla encima: aparece la flecha de volver	Todo lo demás (Panel → Animales → Ficha...)

La navegación relativa funciona como una pila

Panel → Ver Animales → Ficha A123456
 ◀ volver: queda Ver Animales
 ◀ volver: queda Panel

Al cerrar sesión se usa la absoluta para que no quede nada en el historial: si no, alguien podría tocar "volver" y ver datos sin estar logueado.

5.14 Compilación condicional: #if

`#if ANDROID` y `#if DEBUG` son instrucciones para el **compilador**, no condiciones que se evalúan al ejecutar. Al compilar para Android, el compilador se queda solo con el bloque de Android. Van siempre en pareja con `#endif`, como abrir y cerrar llaves.

6. Git y trabajo en equipo

Durante la Fase 0 aparecieron varias situaciones de git que conviene entender, porque se van a repetir.

6.1 Los dos lugares de git

Lugar	Qué es
Tu carpeta (<i>working directory</i>)	Los archivos tal como están ahora en tu compu.
Los commits	"Fotos" guardadas de esos archivos. Cada rama es una serie de fotos.

Al cambiar de rama, git reemplaza los archivos de tu carpeta por los de la última foto de esa rama. Pero si modificaste algo y **no hiciste commit**, git no sabe a qué rama pertenece ese cambio y lo deja en tu carpeta: **el cambio te acompaña a la rama que vayas**.

Nota

Analogía: las ramas son carpetas de un archivero y tu escritorio es uno solo. Si dejás una hoja suelta en el escritorio sin archivarla, sigue ahí aunque abras otra carpeta.

6.2 Leer el git status

Marca	Significa	Ejemplo de la Fase 0
<code>modified / M</code>	Archivo con cambios sin commit	<code>UsuarioResponseDTO.cs</code> con el arreglo del using
<code>Untracked</code>	Archivo que existe en tu carpeta pero git nunca registró	<code>Reportefechasimportantesrepository.cs</code> que no se había subido
<code>deleted</code>	Archivo borrado	Las carpetas de iOS y MacCatalyst
<code>nothing to commit, working tree clean</code>	No hay nada pendiente	—

Regla

Mirá `git status` antes de cada commit. Ahí aparecen los archivos que quedaron afuera o los que se van a colar sin querer.

6.3 Convención de ramas y commits

Prefijo	Para qué	Ejemplo
<code>feature/</code>	Funcionalidad que ve el usuario	<code>feature/login</code>
<code>chore/</code>	Estructura, configuración, mantenimiento	<code>chore/maui-estructura-base</code>
<code>fix/</code>	Arreglar un error	<code>fix/using-rolusuario-usuario-response</code>

Los mensajes de commit siguen la misma idea: `tipo(alcance): descripción`, por ejemplo `chore(maui): handler que agrega el token JWT a cada pedido`.

6.4 El flujo que se usó

Flujo de trabajo

```
# Arrancar una rama desde main actualizado
git checkout main
git pull origin main
git checkout -b chore/maui-estructura-base

# Guardar avances (commits chicos en puntos estables)
git status
git add NutriTrack.MAUI          # solo la carpeta del front, nunca "git add ." a ciegas
git commit -m "chore(maui): ..."
git push -u origin chore/maui-estructura-base # el -u solo la primera vez
git push                          # las siguientes
```

- **git pull antes de crear la rama:** si no, nace desactualizada y aparecen conflictos.
- **Commits chicos en puntos donde todo funciona:** son "puntos de guardado". Si algo se rompe, se vuelve ahí.
- **git add NutriTrack.MAUI** y no **git add .**: así no se cuelan cambios de la API.
- **Push frecuente** como respaldo. Subir una rama no afecta a nadie, porque no toca **main**.
- **Un PR, una sola cosa.** Un arreglo de la API va en su propia rama **fix/**, no en la del front.

6.5 Comandos de rescate

Comando	Para qué
<code>git restore <archivo></code>	Descarta los cambios sin commit de un archivo y lo deja como en el último commit. Es el "deshacer" de git. Borra todos los cambios no commitados de ese archivo.
<code>git diff <archivo></code>	Muestra qué cambiaste (en verde lo agregado, en rojo lo quitado).
<code>git --no-pager diff ...</code>	Igual, pero sin abrir el visor de texto.
<code>q</code>	Salir del visor (<i>pager</i>) cuando git muestra (END) y líneas con <code>~</code> .
<code>git diff --stat</code>	Lista qué archivos cambiaron y cuántas líneas.
<code>git branch -d <rama></code>	Borra una rama local. La <code>-d</code> minúscula se niega si se perderían commits.
<code>git merge main</code>	Trae a tu rama lo nuevo de main (por ejemplo, un fix que se mergeó).

6.6 Abrir un Pull Request

1. En GitHub aparece un cartel con tu rama y el botón **Compare & pull request**. El texto "had recent pushes X hours ago" no siempre se actualiza al momento y no importa: el PR toma todo lo que hay en la rama.
2. Verificá **base:** `main` ← **compare:** **tu rama**.
3. Título con la misma convención que los commits.
4. Descripción: qué se hizo, cómo probarlo, qué quedó sin probar, capturas si es de front y avisos para el revisor (ver Apéndice B).
5. En la pestaña **Commits** del PR deben aparecer todos tus commits.

Tip

Decir qué está verificado y qué no genera confianza en la revisión. Un PR de front con capturas ahorra al revisor tener que correr la app.

7. Fase 0 paso a paso

Este capítulo reconstruye, en orden, todo lo que se hizo en la rama `chore/maui-estructura-base`. Cada paso tiene el objetivo, el porqué, el cómo y la forma de verificar que salió bien. Si estás armando un proyecto parecido, seguilo tal cual.

Paso	Qué se hace	¿Se ve algo?
1-3	Verificar la instalación, crear la rama y correr la plantilla	Sí: el robot
4-5	Paquetes NuGet y plataformas (Windows + Android)	No
6	MVVM y carpetas	No
7-8	Colores del prototipo, tema claro y primer commit	Sí: la app verde
9-10	Conexión con la API (prueba con 200 OK)	Sí: el 200 OK
11	Sesión del usuario (SesionService)	No
12	Handler del token (AuthHeaderHandler)	No
13	Manejo de errores (ResultadoApi + ApiServiceBase)	No
14	BaseViewModel	No
15	Navegación con Shell y pantallas provisionarias	Sí: Login ↔ Panel

Paso 1: verificar que MAUI esté instalado

Visual Studio se instala "por partes" llamadas **workloads**. Programar la API usa una; MAUI usa otra, que trae las herramientas para compilar apps de Windows y Android. Que el proyecto `NutriTrack.MAUI` exista en el repo no significa que tu Visual Studio sepa compilarlo.

PowerShell

```
dotnet workload list
```

Resultado obtenido en la máquina de desarrollo:

Id. de carga de trabajo instalada	Versión del manifiesto	Origen de la instalación
android	36.1.69/10.0.100	VS 18.10.12224.181
ios	26.5.10301/10.0.100	VS 18.10.12224.181
maccatalyst	26.5.10301/10.0.100	VS 18.10.12224.181
maui-windows	10.0.20/10.0.100	VS 18.10.12224.181

- `maui-windows`: para correr en Windows (con lo que se prueba primero).
- `android`: para probar en Android más adelante.
- `ios` y `maccatalyst`: vienen incluidos, pero no se usan.
- "Origen: VS 18.10..." indica que los instaló Visual Studio: las versiones coinciden con el IDE.

Si no aparece `maui` ni `maui-windows`: **Visual Studio Installer → Modificar → tildar "Desarrollo de interfaz de usuario de aplicaciones multiplataforma de .NET" → Modificar.**

Tip

Instalá el workload desde el Installer y no con `dotnet workload install maui`: el Installer garantiza que la versión coincida con tu Visual Studio y evita errores de compilación por desfase de versiones.

No hace falta instalar todavía el emulador de Android (se prueba primero en Windows, que arranca más rápido).

Paso 2: crear la rama

PowerShell (dentro de la carpeta del repo)

```
cd C:\Users\Daniela\source\repos\NutriTrack
git checkout main
git pull origin main
git checkout -b chore/maui-estructura-base
```

- `cd ...`: los comandos de git deben correr **dentro** de la carpeta del repositorio.
- `git checkout main` + `git pull`: arrancar desde lo último que mergeó el equipo.
- `git checkout -b`: crea la rama y se cambia a ella (`-b` = "creala").
- Se usa `chore/` porque es estructura y configuración, no una funcionalidad visible.

Verificación: `git branch` muestra la rama nueva marcada con *. Visual Studio también muestra la rama actual abajo a la derecha.

Atención

Durante este paso apareció `M NutriTrack.API/DT0s/UsuarioResponseDT0.cs` (un cambio sin commitear que "seguía" de rama en rama) y luego un archivo `Untracked` del CU18. Ver los errores 12.2 a 12.4. Finalmente la rama se borró y se volvió a crear desde un `main` actualizado, ya limpia.

Paso 3: correr la plantilla tal como está

1. Clic derecho en `NutriTrack.MAUI` → **Establecer como proyecto de inicio** (queda en negrita).
2. En el desplegable del botón ► elegir **Windows Machine**.
3. Tocar ► (la primera compilación tarda un par de minutos).

Resultado esperado: el robot violeta, "Hello, World!" y el botón "Click me". La app se vio oscura porque Windows estaba en modo oscuro (se corrige en el Paso 7). La barra de íconos arriba al centro es una herramienta de depuración de Visual Studio, no parte de la app.

Por qué este paso

Separar problemas de entorno de problemas de código. Si la plantilla vacía no arranca, el problema es de instalación. Si arranca, cualquier error posterior viene de un cambio nuestro.

Paso 4: instalar los paquetes NuGet

Primero se detiene la app (cuadrado rojo). Mientras corre no se pueden instalar paquetes.

Paquete	Para qué sirve
CommunityToolkit.Mvvm (Microsoft)	Ahorra el código repetitivo de los ViewModels ([ObservableProperty], [RelayCommand]).
Microsoft.Extensions.Http (versión 10.x)	Permite AddHttpClient: configurar la conexión con la API una vez y reutilizarla.

1. Clic derecho en **NutriTrack.MAUI** → **Administrar paquetes NuGet...** → pestaña **Examinar**.
2. Buscar **CommunityToolkit.Mvvm** (autor Microsoft) → **Instalar** con la última estable.
3. Buscar **Microsoft.Extensions.Http** → elegir una versión **10.x** → **Instalar**.

Verificación: en MAUI las dependencias se muestran **por plataforma** (**net10.0-android**, **net10.0-windows...**). Los paquetes están dentro de cada una, en **Paquetes**. La forma más directa es abrir el **.csproj** (doble clic en el proyecto) y buscar las líneas **<PackageReference ...>**.

Paso 5: dejar solo Windows y Android

El documento de requerimientos define "compatible con PC y dispositivos móviles Android". Mantener iOS y Mac hace que Visual Studio prepare cosas que nunca se usan (compila más lento) y además compilar iOS de verdad requiere una Mac.

Antes (.csproj)

```
<TargetFrameworks>net10.0-android</TargetFrameworks>
<TargetFrameworks
  Condition="!$([MSBuild]::IsOSPlatform('linux'))">$(TargetFrameworks);net10.0-ios;net10.0-maccatalyst</TargetFrameworks>
<TargetFrameworks
  Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net10.0-windows10.0.19041.0</TargetFrameworks>
```

Después: se borra SOLO la línea del medio

```
<TargetFrameworks>net10.0-android</TargetFrameworks>
<TargetFrameworks
  Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net10.0-windows10.0.19041.0</TargetFrameworks>
```

La primera línea dice "siempre Android"; la segunda, "si estoy en Windows, agregá Windows". El **Condition** existe porque en Mac o Linux no se puede compilar la versión de Windows.

En el mismo archivo se cambió el nombre visible de la app:

.csproj

```
<ApplicationTitle>NutriTrack</ApplicationTitle>
```

	Dónde está	Para quién	Ejemplo
Nombre técnico	Proyecto, RootNamespace , carpetas	Programadores	NutriTrack.MAUI
Nombre visible	<ApplicationTitle>	El usuario: barra de la ventana en Windows, nombre bajo el ícono en Android	NutriTrack

Cambiar `ApplicationTitle` no toca namespaces, carpetas ni nombres de proyecto. Tampoco es el "Home" que se veía dentro de la app (ese era el título de la página).

Opcional pero prolijo: borrar `Platforms/iOS` y `Platforms/MacCatalyst` (y `Tizen` si existe).

Verificación: en Dependencias quedan solo `net10.0-android` y `net10.0-windows10.0.19041.0`.

Atención

Si se borran las carpetas pero **no** la línea del `.csproj`, el proyecto sigue intentando compilar para iOS sin su punto de entrada y aparece `CS5001: no contiene ningún método Main` al recompilar la solución. Le pasó al equipo: ver error 12.10.

Para la tesis

"Se restringieron los target frameworks a Windows y Android conforme al alcance definido (RNF2, Portabilidad)."

Paso 6: MVVM y carpetas

Con el concepto de MVVM claro (capítulo 5), se crean las carpetas con clic derecho en `NutriTrack.MAUI` → **Agregar** → **Nueva carpeta**:

Carpeta	Qué va a tener
Views	Las pantallas (<code>LoginPage.xaml</code> , <code>PanelPrincipalPage.xaml...</code>)
ViewModels	Un <code>ViewModel</code> por pantalla y el <code>BaseViewModel</code>
Models	Clases de datos (DTOs de la API, <code>SesionUsuario</code> , <code>ResultadoApi</code>)
Services	Comunicación con la API, sesión, handler y configuración
Helpers	Utilidades compartidas (converters, etc.)

Nota

Git **no guarda carpetas vacías**, solo archivos. Las carpetas aparecen en `git status` recién cuando tienen al menos un archivo adentro.

Paso 7: colores del prototipo y tema claro

7.1 La paleta (Colors.xaml)

Se reemplazaron los seis colores violetas por los verdes del prototipo y se agregaron colores propios con **nombres según su uso** (`Peligro` y no `Rojo`): así el XAML se lee "este texto es de peligro", y si mañana el peligro pasa a ser naranja, el nombre sigue teniendo sentido. Los nombres originales (`Primary`, `Secondary...`) se mantuvieron porque `Styles.xaml` los usa en muchos lugares.

Resources/Styles/Colors.xml (archivo completo)

```
<?xml version="1.0" encoding="UTF-8" ?>
<ResourceDictionary
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml">

    <!-- Note: For Android please see also Platforms\Android\Resources\values\colors.xml -->

    <!-- Paleta NutriTrack (tomada del prototipo de Figma) -->
    <Color x:Key="Primary">#00A63E</Color>
    <Color x:Key="PrimaryDark">#008236</Color>
    <Color x:Key="PrimaryDarkText">#242424</Color>
    <Color x:Key="Secondary">#F0FDF4</Color>
    <Color x:Key="SecondaryDarkText">#008236</Color>
    <Color x:Key="Tertiary">#016630</Color>

    <!-- Colores propios de NutriTrack -->
    <Color x:Key="Fondo">#F9FAFB</Color>
    <Color x:Key="Superficie">#FFFFFF</Color>
    <Color x:Key="Borde">#E5E7EB</Color>
    <Color x:Key="TextoPrincipal">#101828</Color>
    <Color x:Key="TextoSecundario">#6A7282</Color>
    <Color x:Key="Peligro">#E7000B</Color>
    <Color x:Key="PeligroSuave">#FEF2F2</Color>

    <!-- Colores base que usa Styles.xml (NO BORRAR) -->
    <Color x:Key="White">White</Color>

    <Color x:Key="Black">Black</Color>
    <Color x:Key="Magenta">#D600AA</Color>
    <Color x:Key="MidnightBlue">#190649</Color>
    <Color x:Key="OffBlack">#1f1f1f</Color>

    <Color x:Key="Gray100">#E1E1E1</Color>
    <Color x:Key="Gray200">#C8C8C8</Color>
    <Color x:Key="Gray300">#ACACAC</Color>
    <Color x:Key="Gray400">#919191</Color>
    <Color x:Key="Gray500">#6E6E6E</Color>
    <Color x:Key="Gray600">#404040</Color>
    <Color x:Key="Gray900">#212121</Color>
    <Color x:Key="Gray950">#141414</Color>

    <SolidColorBrush x:Key="PrimaryBrush" Color="{StaticResource Primary}"/>
    <SolidColorBrush x:Key="SecondaryBrush" Color="{StaticResource Secondary}"/>
    <SolidColorBrush x:Key="TertiaryBrush" Color="{StaticResource Tertiary}"/>
    <SolidColorBrush x:Key="WhiteBrush" Color="{StaticResource White}"/>
    <SolidColorBrush x:Key="BlackBrush" Color="{StaticResource Black}"/>

    <SolidColorBrush x:Key="Gray100Brush" Color="{StaticResource Gray100}"/>
    <SolidColorBrush x:Key="Gray200Brush" Color="{StaticResource Gray200}"/>
    <SolidColorBrush x:Key="Gray300Brush" Color="{StaticResource Gray300}"/>
    <SolidColorBrush x:Key="Gray400Brush" Color="{StaticResource Gray400}"/>
    <SolidColorBrush x:Key="Gray500Brush" Color="{StaticResource Gray500}"/>
    <SolidColorBrush x:Key="Gray600Brush" Color="{StaticResource Gray600}"/>

    <SolidColorBrush x:Key="Gray900Brush" Color="{StaticResource Gray900}"/>
    <SolidColorBrush x:Key="Gray950Brush" Color="{StaticResource Gray950}"/>
</ResourceDictionary>
```

Atención

Todo lo que está debajo del comentario "NO BORRAR" lo usa **Styles.xml**. Si falta **White** (u otro), la app compila pero explota al abrir con **StaticResource not found for key 'White'**. Ver error 12.8.

Los colores se tomaron a ojo de las capturas del prototipo. Para valores exactos: en Figma, seleccionar un elemento (por ejemplo el botón "Ingresar") y leer **Fill** en el panel derecho.

7.2 Tema claro fijo

Opción	Pro	Contra
Diseñar también un tema oscuro	Más moderno	El doble de diseño y no está en el prototipo
Fijar siempre el tema claro	Igual en todas las compus, coincide con el prototipo	No respeta la preferencia del sistema

App.xaml.cs (primera versión; la final está en el Paso 15)

```
public App()
{
    InitializeComponent();
    UserAppTheme = AppTheme.Light;
}
```

Para la tesis

"Se fijó el tema claro para garantizar consistencia visual con el prototipo validado (RNF1, Usabilidad)."

7.3 Ícono, pantalla de carga y Android

El ícono y la pantalla de carga se generan al compilar, antes de que la app lea `Colors.xaml`, así que su color se define en el `.csproj`:

.csproj (fragmentos)

```
<!-- Nombre visible de la app (barra de la ventana / nombre bajo el ícono en Android) -->
<ApplicationTitle>NutriTrack</ApplicationTitle>

<!-- Ícono y pantalla de carga con el verde de NutriTrack -->
<MauiIcon Include="Resources\AppIcon\appicon.svg" ForegroundFile="Resources\AppIcon\appiconfg.svg"
    Color="#00A63E" />
<MauiSplashScreen Include="Resources\Splash\splash.svg" Color="#00A63E" BaseSize="128,128" />

<!-- Paquetes NuGet agregados (las versiones pueden variar) -->
<PackageReference Include="CommunityToolkit.Mvvm" Version="..." />
<PackageReference Include="Microsoft.Extensions.Http" Version="10..." />
```

Y Android usa su propio archivo de colores para la barra de estado del celular:

Platforms/Android/Resources/values/colors.xml (archivo completo)

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#00A63E</color>
    <color name="colorPrimaryDark">#008236</color>
    <color name="colorAccent">#008236</color>
</resources>
```

Archivo	Ruta	Formato	Cómo reconocerlo
Colors.xaml	Resources/Styles/	XAML de MAUI	<ResourceDictionary>, <Color x:Key="...">
colors.xml	Platforms/Android/Resources/values/	XML de Android	<resources>, <color name="...">

Atención

El equipo confundió estos dos archivos y pegó la paleta de MAUI en el de Android. Truco: `Colors.xaml` usa `x:Key` y `<Color>` con mayúscula; el de Android usa `name` y `<color>` con minúscula. Ver error 12.9.

Verificación: fondo blanco aunque Windows esté en oscuro y botón "Click me" verde. El robot sigue violeta porque es una imagen.

Paso 8: primer commit

PowerShell

```
git status
git add NutriTrack.MAUI
git commit -m "chore(maui): paquetes MVVM/Http, targets Windows y Android, paleta NutriTrack y tema claro"
git push -u origin chore/maui-estructura-base
```

El `git status` debe mostrar solo archivos de `NutriTrack.MAUI/`. El push sube la rama como **respaldo**; el PR se abre recién al terminar la Fase 0.

Paso 9: conectar el front con la API

La app necesita saber tres cosas: **a qué dirección** mandar los pedidos, **con qué herramienta** (`HttpClient`) y configurarlo **una sola vez**.

9.1 La dirección de la API (ApiConfig)

La API con el perfil **http** escucha en `http://localhost:5068` (dato de `launchSettings.json`). Pero "localhost" significa "esta misma máquina" y eso cambia según dónde corra la app:

Dónde corre la app	Qué es "localhost" para ella	Dirección correcta
Windows (tu PC)	Tu PC, donde está la API	<code>http://localhost:5068</code>
Emulador de Android	El propio emulador (otra "computadora")	<code>http://10.0.2.2:5068</code>

Services/ApiConfig.cs

```
namespace NutriTrack.MAUI.Services
{
    public static class ApiConfig
    {
        #if ANDROID
            // El emulador de Android llega a la PC a través de 10.0.2.2
            // ("localhost" dentro del emulador sería el propio emulador).
            public const string BaseUrl = "http://10.0.2.2:5068/";
        #else
            public const string BaseUrl = "http://localhost:5068/";
        #endif
    }
}
```

La dirección está en un solo lugar: si la API se publica en un servidor, se cambia acá.

9.2 Registrar el HttpClient

MauiProgram.cs (fragmento; la versión final está en el Paso 15)

```
builder.Services.AddHttpClient("NutriTrackApi", client =>
{
    client.BaseAddress = new Uri(ApiConfig.BaseUrl);
    client.Timeout = TimeSpan.FromSeconds(15);
});
```

- **"NutriTrackApi"**: nombre del cliente. Los servicios lo piden por ese nombre y lo reciben configurado.

- **BaseUrl**: cada pedido indica solo el final ("**api/login**") y se arma la URL completa.
- **Timeout** de 15 segundos: si la API no contesta, se corta y se muestra un error en lugar de dejar la app colgada. El RNF3 pide 2 a 5 segundos; 15 es margen para una red de campo lenta.
- No se hace `new HttpClient()` en cada pantalla porque crear y descartar clientes puede agotar las conexiones del sistema. **AddHttpClient** las reutiliza de forma segura.

9.3 Prueba de conexión (descartable)

Código temporal en el botón de la plantilla

```
// PRUEBA TEMPORAL: no commitear (MainPage.xaml.cs de la plantilla)
using Microsoft.Extensions.DependencyInjection;

private async void OnCounterClicked(object? sender, EventArgs e)
{
    var factory = IPlatformApplication.Current!.Services.GetRequiredService<IHttpClientFactory>();
    var http = factory.CreateClient("NutriTrackApi");

    try
    {
        var respuesta = await http.GetAsync("api/Ingrediente");
        CounterBtn.Text = $"Respuesta: {(int)respuesta.StatusCode} {respuesta.StatusCode}";
    }
    catch (Exception ex)
    {
        CounterBtn.Text = $"Sin conexión: {ex.Message}";
    }
}
```

9.4 Correr la API y la app al mismo tiempo

Se intentó con **Varios proyectos de inicio** en Visual Studio y aparecieron dos errores (12.5 y 12.6). La forma que quedó como recomendada es **la API en una terminal y la app desde Visual Studio**:

PowerShell aparte (fuera de Visual Studio)

```
cd C:\Users\Daniela\source\repos\NutriTrack
dotnet run --project NutriTrack.API --launch-profile http
# esperar: "Now listening on: http://localhost:5068" (dejar la terminal abierta; Ctrl + C la apaga)
```

- Visual Studio con **Proyecto de inicio único**: **NutriTrack.MAUI**, y ► con Windows Machine.
- Ventaja: se reinicia la app sin reiniciar la API, y se ven los logs de la API en vivo.
- Se usa el perfil **http** y no https: con https la API redirige los pedidos y en Android hay problemas con el certificado de desarrollo.
- La base de datos tiene que estar levantada en Docker.

9.5 Cómo leer el resultado

El botón muestra	Significa
Respuesta: 200 OK	La app llegó a la API y la API respondió bien. Resultado obtenido.
Respuesta: 401 Unauthorized	La app llegó; la API pide login (después del PR del login). La conexión anda.
Respuesta: 500 ...	La app llegó; la API falló por dentro (casi siempre la base apagada).
Sin conexión: ...	La app no llegó: la API no corre o está en otro puerto.

Para la tesis

"Se validó la comunicación cliente-servidor mediante una petición GET a `api/Ingrediente` desde la aplicación MAUI, obteniendo respuesta HTTP 200."

Paso 10: quitar la prueba y guardar

PowerShell

```
git restore NutriTrack.MAUI/MainPage.xaml.cs # deshace la prueba temporal
git status
git add NutriTrack.MAUI
git commit -m "chore(maui): configurar HttpClient y URL base de la API por plataforma"
git push
```

En el `git status` no debe aparecer `MainPage.xaml.cs`. Si aparece `NutriTrack.API/DT0s/UsuarioResponseDT0.cs` como modificado, es un arreglo local que no va en este PR (ver error 12.4) y no se agrega porque se usa `git add NutriTrack.MAUI`.

Paso 11: la sesión del usuario

La respuesta del login (token, expiración, nombre de usuario y rol) se necesita en toda la app: el token para cada pedido, el rol para decidir qué mostrar y la expiración para saber si la sesión sigue vigente. Hace falta **un único lugar** donde vivan esos datos: el `SesionService`.

Decisión: dónde guardar el token

Opción	Cómo funciona	Pro	Contra
Solo en memoria	Una variable	Simple	Al cerrar la app hay que volver a loguearse
Preferences	Disco, texto plano	Sobrevive al cierre	Cualquiera con acceso al archivo lee el token
SecureStorage	Disco, cifrado por el sistema operativo	Sobrevive al cierre y está protegido	Más lento (por eso se combina con memoria)

Se eligió **SecureStorage + copia en memoria**, y además **recordar la sesión**: si al abrir la app hay un token guardado y vigente, se va directo al panel (mejor para el encargado de campo que la usa todo el día).

Para la tesis

"El token de acceso se persiste cifrado mediante SecureStorage (RNF4, Seguridad)."

Models/RolUsuario.cs

```
namespace NutriTrack.MAUI.Models
{
    // IMPORTANTE: los nombres tienen que coincidir EXACTAMENTE con
    // NutriTrack.Core.Entities.Enums.RolUsuario, porque la API los
    // manda como texto ("Administrador", "EncargadoDeCampo"...).
    public enum RolUsuario
    {
        AsesorTecnico,
        EncargadoDeCampo,
        Administrador
    }
}
```

Se copia el enum en vez de referenciar **NutriTrack.Core**, porque Core trae también las entidades de la base y el front no debería conocerlas: solo habla con la API mediante DTOs. El riesgo (que alguien cambie el enum del back y se olvide del front) se cubre con el comentario.

Models/SesionUsuario.cs

```
namespace NutriTrack.MAUI.Models
{
    // Datos del usuario logueado. Se arman a partir de la respuesta
    // de POST api/login y se guardan mientras la sesión esté vigente.
    public class SesionUsuario
    {
        public string Token { get; set; } = string.Empty;
        public DateTime Expiracion { get; set; }
        public string NombreUsuario { get; set; } = string.Empty;
        public RolUsuario Rol { get; set; }

        public bool EstaVigente => Expiracion > DateTime.UtcNow;
    }
}
```

Atención

Se compara con **DateTime.UtcNow** y no con **DateTime.Now** porque el back genera la expiración en UTC. Comparar UTC con la hora argentina (UTC−3) haría que la sesión pareciera durar 3 horas más o menos de lo real.

Services/ISesionService.cs (interfaz: Agregar → Nuevo elemento → Interfaz)

```
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    public interface ISesionService
    {
        // Datos del usuario logueado (null si no hay sesión).
        SesionUsuario? SesionActual { get; }

        bool HaySesionActiva { get; }

        // Se dispara cuando la API rechaza el token (vencido o inválido).
        event EventHandler? SesionExpirada;

        Task IniciarSesionAsync(SesionUsuario sesion);
        Task<bool> RestaurarSesionAsync();
        void CerrarSesion();
        void NotificarSesionExpirada();
    }
}
```

Services/SesionService.cs

```
using System.Text.Json;
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    public class SesionService : ISesionService
    {
        private const string ClaveAlmacenamiento = "sesion_nutritrack";

        public SesionUsuario? SesionActual { get; private set; }

        public bool HaySesionActiva => SesionActual is { EstaVigente: true };

        public event EventHandler? SesionExpirada;

        public async Task IniciarSesionAsync(SesionUsuario sesion)
        {
            SesionActual = sesion;

            try
            {
                var json = JsonSerializer.Serialize(sesion);
                await SecureStorage.Default.SetAsync(ClaveAlmacenamiento, json);
            }
            catch (Exception)
            {
                // Si el almacenamiento seguro falla, la sesión sigue
                // funcionando en memoria; solo no se recordará al reabrir.
            }
        }

        public async Task<bool> RestaurarSesionAsync()
        {
            try
            {
                var json = await SecureStorage.Default.GetAsync(ClaveAlmacenamiento);
                if (string.IsNullOrEmpty(json))
                    return false;

                var sesion = JsonSerializer.Deserialize<SesionUsuario>(json);
                if (sesion is null || !sesion.EstaVigente)
                {
                    CerrarSesion();
                    return false;
                }

                SesionActual = sesion;
                return true;
            }
            catch (Exception)
            {
                CerrarSesion();
            }
        }
    }
}
```

```

        return false;
    }
}

public void CerrarSesion()
{
    SesionActual = null;
    SecureStorage.Default.Remove(ClaveAlmacenamiento);
}

public void NotificarSesionExpirada()
{
    CerrarSesion();
    SesionExpirada?.Invoke(this, EventArgs.Empty);
}
}
}

```

Método	Cuándo se usa	Qué hace
IniciarSesionAsync	Después de un login exitoso	Guarda la sesión en memoria y cifrada en disco
RestaurarSesionAsync	Al abrir la app	Recupera una sesión guardada si existe y no venció
CerrarSesion	Botón "Cerrar sesión"	Borra todo, de memoria y de disco
NotificarSesionExpirada	Cuando la API devuelve 401	Cierra la sesión y dispara el evento <code>SesionExpirada</code>

- **Encapsulamiento:** { `get`; `private set`; } permite leer la sesión desde cualquier lado, pero solo el servicio la puede cambiar.
- **Degradar con elegancia:** los `try/catch` hacen que, si el almacenamiento seguro falla, la sesión funcione igual en memoria. La app nunca debería crashear por no poder recordar algo.
- **El evento no navega:** el servicio solo avisa "se venció la sesión". Quien escucha (el Shell) decide qué hacer. El servicio no depende de la interfaz.

MauiProgram.cs

```
// Sesión del usuario: Singleton porque tiene que ser UNA sola instancia compartida por toda la app
builder.Services.AddSingleton<ISesionService, SesionService>();
```

Verificación: compilar el proyecto MAUI sin errores. En este paso aparecieron los errores 12.10 y 12.11.

Paso 12: el handler del token

Con el login activo, la API rechaza con 401 cualquier pedido sin token. Que cada servicio agregue el encabezado a mano significa decenas de lugares y alcanza con que alguien se olvide en uno. Un **handler** se mete en el medio de **todos** los pedidos y lo agrega solo.

Services/AuthHeaderHandler.cs

```

using System.Net;
using System.Net.Http.Headers;

namespace NutriTrack.MAUI.Services
{
    // Se ejecuta en TODOS los pedidos del cliente "NutriTrackApi":
    // 1) Si hay sesión, agrega el token en el encabezado Authorization.
    // 2) Si la API responde 401 teniendo sesión, avisa que la sesión expiró.
    public class AuthHeaderHandler : DelegatingHandler
    {
        private readonly ISesionService _sesionService;

        public AuthHeaderHandler(ISesionService sesionService)
        {
            _sesionService = sesionService;
        }

        protected override async Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            var sesion = _sesionService.SesionActual;

            if (sesion is not null)
            {
                request.Headers.Authorization =
                    new AuthenticationHeaderValue("Bearer", sesion.Token);
            }

            // Acá el pedido sale realmente hacia la API y esperamos la respuesta.
            var respuesta = await base.SendAsync(request, cancellationToken);

            if (respuesta.StatusCode == HttpStatusCode.Unauthorized && sesion is not null)
            {
                _sesionService.NotificarSesionExpirada();
            }

            return respuesta;
        }
    }
}

```

Se lee como una historia en tres momentos: **antes de enviar** (agrega el token), **base.SendAsync** (el pedido sale de verdad) y **después de recibir** (si fue 401, avisa).

La condición clave: && sesion is not null

Situación	¿Había sesión?	401 significa	Qué se hace
Contraseña incorrecta en el login	No	Credenciales inválidas	Nada acá; el login muestra el error
Pedido normal con token vencido	Sí	El token ya no sirve	Cerrar sesión y volver al login

Regla

El mismo código de error puede significar cosas distintas según el contexto. Pensar esos casos borde es parte del trabajo de ingeniería.

MauiProgram.cs

```
builder.Services.AddTransient<AuthHeaderHandler>();

builder.Services.AddHttpClient("NutriTrackApi", client => { ... })
    .AddHttpMessageHandler<AuthHeaderHandler>();
```

- `.AddHttpMessageHandler<AuthHeaderHandler>()`: todo pedido del cliente pasa por el handler.
- El handler se registra como **Transient** (obligatorio: el sistema de `HttpClient` los crea y recicla).
- Aunque haya muchos handlers, todos reciben el **mismo** `SesionService` singleton.

Paso 13: manejo de errores de la API

Situación	Qué llega	Ejemplo en NutriTrack
Todo bien	200 + JSON	La lista de animales
Error de negocio	400 + texto del controller	"La caravana ya existe" (CU1, E1)
Validación automática	400 + JSON de ASP.NET con "errors"	Login con un campo vacío
No encontrado	404 + texto	"No se encontró un animal con esa caravana"
Sin permiso / token vencido	401 o 403	Token vencido
Error del servidor	500	Base caída
Sin conexión	Excepción <code>HttpRequestException</code>	API apagada o sin internet en el campo
Timeout	Excepción <code>TaskCanceledException</code>	Red lenta (más de 15 segundos)

Models/ResultadoApi.cs

```
using System.Net;

namespace NutriTrack.MAUI.Models
{
    // Resultado de cualquier pedido a la API.
    // Si Exito es true, los datos están en Datos.
    // Si es false, MensajeError trae un texto listo para mostrar al usuario.
    public class ResultadoApi<T>
    {
        public bool Exito { get; private init; }
        public T? Datos { get; private init; }
        public string? MensajeError { get; private init; }
        public HttpStatusCode?CodigoEstado { get; private init; }

        public static ResultadoApi<T> Ok(T? datos) =>
            new() { Exito = true, Datos = datos };

        public static ResultadoApi<T> Error(string mensaje, HttpStatusCode? codigo = null) =>
            new() { Exito = false, MensajeError = mensaje, CodigoEstado = codigo };
    }
}
```

- `<T>` (genérico): una sola clase sirve para todos los pedidos (`ResultadoApi<List<AnimalDto>>`, `ResultadoApi<LoginResponse>`...).

- `Ok(...)` y `Error(...)` son las únicas formas de crearlo: nunca existe un resultado "a medias".
- `private init`: una vez creado nadie lo cambia.

Services/ApiServiceBase.cs

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using System.Text.Json.Serialization;
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    // Clase base para todos los servicios que hablan con la API
    // (AuthService, AnimalService, RodeoService...).
    // Centraliza el manejo de errores para que ninguna pantalla tenga que hacerlo.
    public abstract class ApiServiceBase
    {
        protected readonly HttpClient Http;

        // Mismas reglas de JSON que la API: camelCase y enums como texto.
        protected static readonly JsonSerializerOptions OpcionesJson =
            new(JsonSerializerDefaults.Web)
            {
                Converters = { new JsonStringEnumConverter() }
            };

        protected ApiServiceBase(IHttpClientFactory httpClientFactory)
        {
            Http = httpClientFactory.CreateClient("NutriTrackApi");
        }

        // ----- Pedidos que devuelven datos -----

        protected Task<ResultadoApi<T>> GetAsync<T>(string url) =>
            EnviarAsync<T>(() => Http.GetAsync(url), leerDatos: true);

        protected Task<ResultadoApi<T>> PostAsync<T>(string url, object cuerpo) =>
            EnviarAsync<T>(() => Http.PostAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos: true);

        protected Task<ResultadoApi<T>> PutAsync<T>(string url, object cuerpo) =>
            EnviarAsync<T>(() => Http.PutAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos: true);

        // ----- Pedidos donde solo importa si salió bien -----

        protected Task<ResultadoApi<bool>> PostSinRespuestaAsync(string url, object cuerpo) =>
            EnviarAsync<bool>(() => Http.PostAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos:
                false);

        protected Task<ResultadoApi<bool>> PutSinRespuestaAsync(string url, object cuerpo) =>
            EnviarAsync<bool>(() => Http.PutAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos:
                false);

        protected Task<ResultadoApi<bool>> DeleteAsync(string url) =>
            EnviarAsync<bool>(() => Http.DeleteAsync(url), leerDatos: false);

        // ----- El corazón: un solo lugar que maneja todos los casos -----
    }
}
```

```

private async Task<ResultadoApi<T>> EnviarAsync<T>(
    Func<Task<HttpResponseMessage>> pedido, bool leerDatos)
{
    try
    {
        using var respuesta = await pedido();

        if (!respuesta.IsSuccessStatusCode)
        {
            var mensaje = await LeerMensajeErrorAsync(respuesta);
            return ResultadoApi<T>.Error(mensaje, respuesta.StatusCode);
        }

        if (!leerDatos)
            return ResultadoApi<T>.Ok(default);

        // Algunos endpoints devuelven un texto plano en vez de JSON.
        if (typeof(T) == typeof(string))
        {
            var texto = await respuesta.Content.ReadAsStringAsync();
            return ResultadoApi<T>.Ok((T)(object)texto);
        }

        var datos = await respuesta.Content.ReadFromJsonAsync<T>(OpcionesJson);
        return ResultadoApi<T>.Ok(datos);
    }
    catch (TaskCanceledException)
    {
        return ResultadoApi<T>.Error(
            "El servidor tardó demasiado en responder. Intentá de nuevo.");
    }
    catch (HttpRequestException)
    {
        return ResultadoApi<T>.Error(
            "No se pudo conectar con el servidor. Verificá tu conexión a internet.");
    }
    catch (JsonException)
    {
        return ResultadoApi<T>.Error(
            "La respuesta del servidor no tiene el formato esperado.");
    }
}

// Convierte la respuesta de error de la API en un mensaje para el usuario.
private static async Task<string> LeerMensajeErrorAsync(HttpResponseMessage respuesta)
{
    // Errores 500: nunca mostramos el detalle interno del servidor.
    if ((int)respuesta.StatusCode >= 500)
        return MensajePorDefecto(respuesta.StatusCode);

    var contenido = (await respuesta.Content.ReadAsStringAsync()).Trim();

```

```

        if (string.IsNullOrEmpty(contenido))
            return MensajePorDefecto(respuesta.StatusCode);

        // Caso 1: errores de validación automáticos de ASP.NET (JSON con "errors").
        if (contenido.StartsWith('{'))
        {
            try
            {
                using var documento = JsonDocument.Parse(contenido);
                if (documento.RootElement.TryGetProperty("errors", out var errores))
                {
                    foreach (var campo in errores.EnumerateObject())
                        foreach (var detalle in campo.Value.EnumerateArray())
                            return detalle.GetString() ??
                                MensajePorDefecto(respuesta.StatusCode);
                }
            }
            catch (JsonException) { }

            return MensajePorDefecto(respuesta.StatusCode);
        }

        // Caso 2: mensajes de texto de los controllers, ej. BadRequest("La caravana ya
        // existe").
        return contenido.Trim('');
    }

    private static string MensajePorDefecto(HttpStatusCode codigo) => codigo switch
    {
        HttpStatusCode.BadRequest => "Los datos enviados no son válidos.",
        HttpStatusCode.Unauthorized => "Tu sesión expiró. Volvé a iniciar sesión.",
        HttpStatusCode.Forbidden => "No tenés permisos para realizar esta acción.",
        HttpStatusCode.NotFound => "No se encontró lo que buscabas.",
        _ when (int)codigo >= 500 => "Ocurrió un error en el servidor. Intentá de nuevo más
            tarde.",
        _ => "Ocurrió un error inesperado."
    };
}
}

```

- **abstract**: no se usa sola, se hereda. Cada servicio (**AuthService**, **AnimalService**...) hereda todo el manejo de errores.
- **EnviarAsync** es el corazón: recibe el pedido como función (**Func<Task<HttpResponseMessage>>**) y lo envuelve en el **try/catch**. El manejo de errores está escrito una sola vez.
- Los mensajes de los controllers (**BadRequest("...")**) llegan como texto plano y se muestran tal cual: los mensajes escritos en los CU le llegan al usuario sin tocarlos.
- **Los 500 nunca muestran el detalle**: puede incluir rutas o nombres de tablas. Es feo para el usuario y da pistas a un atacante.
- **OpcionesJson** replica las reglas de la API: camelCase y enums como texto. Sin eso, el rol del login no se podría leer.

Con esta base, un servicio nuevo es una línea:

Ejemplo

```

public Task<ResultadoApi<List<AnimalDto>>> ObtenerAnimalesAsync() =>
    GetAsync<List<AnimalDto>>("api/Animal");

```

Paso 14: el BaseViewModel

Casi todas las pantallas hacen lo mismo al hablar con la API: el usuario toca un botón, se muestra "cargando" y se desactiva el botón, se llama a la API, si falla se muestra un error, se quita el "cargando". El `BaseViewModel` lo resuelve una vez para todos.

ViewModels/BaseViewModel.cs

```
using CommunityToolkit.Mvvm.ComponentModel;

namespace NutriTrack.MAUI.ViewModels
{
    // Clase base de todos los ViewModels de NutriTrack.
    // Centraliza el estado "cargando" y el mensaje de error que comparten las pantallas.
    public partial class BaseViewModel : ObservableObject
    {
        // true mientras se espera una respuesta de la API.
        [ObservableProperty]
        [NotifyPropertyChangedFor(nameof(NoEstaOcupado))]
        public partial bool EstaOcupado { get; set; }

        // Lo contrario de EstaOcupado, para habilitar botones desde el XAML.
        public bool NoEstaOcupado => !EstaOcupado;

        // Mensaje de error para mostrar en pantalla (null si no hay error).
        [ObservableProperty]
        [NotifyPropertyChangedFor(nameof(HayError))]
        public partial string? MensajeError { get; set; }

        public bool HayError => !string.IsNullOrEmpty(MensajeError);

        // Ejecuta una operación mostrando "cargando" y limpiando errores anteriores.
        // Uso en cualquier ViewModel: await EjecutarAsync(async () => { ... });
        protected async Task EjecutarAsync(Func<Task> operacion)
        {
            if (EstaOcupado)
                return;

            try
            {
                EstaOcupado = true;
                MensajeError = null;
                await operacion();
            }
            finally
            {
                EstaOcupado = false;
            }
        }
    }
}
```

- `[NotifyPropertyChangedFor(nameof(NoEstaOcupado))]`: `NoEstaOcupado` depende de `EstaOcupado`; el atributo avisa a la pantalla que también cambió.
- Las propiedades "al revés" existen porque el XAML no puede negar valores (no existe `{Binding !EstaOcupado}`).
- `if (EstaOcupado) return;` evita el **doble envío**: en el CU13 un doble toque registraría el evento sanitario dos veces.
- `MensajeError = null;` borra el error del intento anterior.
- `finally` se ejecuta siempre: sin él, ante una falla la ruedita quedaría girando para siempre.

Cómo se usa desde el XAML

```
<Button Text="Ingresar" IsEnabled="{Binding NoEstaOcupado}" />
<ActivityIndicator IsRunning="{Binding EstaOcupado}" />
<Label Text="{Binding MensajeError}" IsVisible="{Binding HayError}" />
```

Paso 15: navegación con Shell y pantallas provisionarias

15.1 Crear las pantallas

Clic derecho en **Views** → **Agregar** → **Nuevo elemento...** Si se abre la ventana compacta, tocar **Mostrar todas las plantillas**. En el buscador escribir **ContentPage** y elegir **.NET MAUI ContentPage (XAML)** (no la que dice C#).

Views/LoginPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="NutriTrack.MAUI.Views.LoginPage"
              Shell.NavBarIsVisible="False"
              BackgroundColor="{StaticResource Fondo}">

    <!-- PROVISORIA: el diseño real se hace en la Fase 1 -->
    <VerticalStackLayout Spacing="16" VerticalOptions="Center" HorizontalOptions="Center">
        <Label Text="NutriTrack"
              FontSize="32"
              FontAttributes="Bold"
              TextColor="{StaticResource TextoPrincipal}"
              HorizontalOptions="Center" />
        <Label Text="Pantalla de login (en construcción)"
              TextColor="{StaticResource TextoSecundario}"
              HorizontalOptions="Center" />
        <Button Text="Entrar (prueba de navegación)"
              Clicked="OnEntrarClickado" />
    </VerticalStackLayout>
</ContentPage>
```

Views/LoginPage.xaml.cs

```
namespace NutriTrack.MAUI.Views
{
    public partial class LoginPage : ContentPage
    {
        public LoginPage()
        {
            InitializeComponent();
        }

        // PROVISORIO: en la Fase 1 esto lo reemplaza el LoginViewModel.
        private async void OnEntrarClickado(object? sender, EventArgs e)
        {
            await Shell.Current.GoToAsync("//panel");
        }
    }
}
```

Views/PanelPrincipalPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="NutriTrack.MAUI.Views.PanelPrincipalPage"
              Title="Panel Principal"
              BackgroundColor="{StaticResource Fondo}">

    <!-- PROVISORIA: el diseño real se hace en la Fase 2 -->
    <VerticalStackLayout Spacing="16" VerticalOptions="Center" HorizontalOptions="Center">
        <Label Text="Panel principal (en construcción)"
              TextColor="{StaticResource TextoSecundario}"
              HorizontalOptions="Center" />
        <Button Text="Cerrar sesión (prueba de navegación)"
              Clicked="OnCerrarSesionClicked" />
    </VerticalStackLayout>
</ContentPage>
```

Views/PanelPrincipalPage.xaml.cs

```
namespace NutriTrack.MAUI.Views
{
    public partial class PanelPrincipalPage : ContentPage
    {
        public PanelPrincipalPage()
        {
            InitializeComponent();

            // PROVISORIO: en la Fase 1 esto también va a cerrar la sesión real.
            private async void OnCerrarSesionClicked(object? sender, EventArgs e)
            {
                await Shell.Current.GoToAsync("//login");
            }
        }
    }
}
```

- `Shell.NavBarIsVisible="False"`: el login no tiene barra arriba (como en el prototipo).
- `{StaticResource Fondo}` y `TextoSecundario`: primer uso de los colores propios.

Atención

Al reemplazar el contenido de un archivo, usar siempre **Ctrl + A** y después pegar. Si solo se pega arriba, lo viejo queda abajo y aparece `Data at the root level is invalid` (error 12.13).

15.2 El mapa de la app

AppShell.xaml

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
    x:Class="NutriTrack.MAUI.AppShell"
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:views="clr-namespace:NutriTrack.MAUI.Views"
    Shell.FlyoutBehavior="Disabled"
    Shell.BackgroundColor="{StaticResource Primary}"
    Shell.ForegroundColor="White"
    Shell.TitleColor="White"
    Title="NutriTrack">

    <!-- Pantallas principales: se navega a ellas con "//ruta" (borra el historial) -->
    <ShellContent Route="login"
        ContentTemplate="{DataTemplate views:LoginPage}" />

    <ShellContent Route="panel"
        Title="Panel Principal"
        ContentTemplate="{DataTemplate views:PanelPrincipalPage}" />
</Shell>
```

- `xmlns:views="clr-namespace:NutriTrack.MAUI.Views"`: es como un `using` del XAML.
- Cada `ShellContent` es una pantalla raíz con su `Route`. **La primera es la que aparece al abrir la app**; por eso el login va primero.
- `FlyoutBehavior="Disabled"`: sin menú hamburguesa; se navega con los botones del panel.
- Los tres colores pintan la barra de verde con texto blanco, como el prototipo.

15.3 Recordar la sesión y reaccionar cuando expira**AppShell.xaml.cs**

```
using NutriTrack.MAUI.Services;

namespace NutriTrack.MAUI
{
    public partial class AppShell : Shell
    {
        private readonly ISessionService _sessionService;

        public AppShell(ISessionService sessionService)
        {
            InitializeComponent();

            // Acá se registran las pantallas que se apilan (navegación relativa), por ejemplo:
            // Routing.RegisterRoute("animales", typeof(AnimalesPage));

            _sessionService = sessionService;
            _sessionService.SesionExpirada += OnSesionExpirada;

            Loaded += OnLoaded;
        }

        // Al abrir la app: si hay una sesión guardada y vigente, ir directo al panel.
        private async void OnLoaded(object? sender, EventArgs e)
        {
            if (await _sessionService.RestaurarSesionAsync())
                await GoToAsync("//panel");
        }
    }
}
```

```

    }

    // La API rechazó el token (vencido o inválido): avisar y volver al login.
    private void OnSesionExpirada(object? sender, EventArgs e)
    {
        MainThread.BeginInvokeOnMainThread(async () =>
        {
            await DisplayAlertAsync("Sesión expirada",
                                   "Tu sesión expiró. Volvé a iniciar sesión.", "Aceptar");
            await GoToAsync("//login");
        });
    }
}

```

- El Shell recibe el `ISesionService` por constructor (la forma correcta).
- `+= OnSesionExpirada`: se suscribe al evento del servicio.
- `MainThread.BeginInvokeOnMainThread`: el aviso puede llegar desde un hilo secundario y solo el principal puede mostrar alertas y navegar.
- Si `DisplayAlertAsync` aparece en rojo, usar `DisplayAlert` (el nombre cambió en las versiones nuevas de MAUI).

15.4 App.xaml.cs: crear el Shell en el momento correcto

La primera versión de este paso hacía que `App` recibiera el `AppShell` por constructor. Eso produjo `StaticResource not found for key 'Primary'` por un problema de **orden de ejecución** (error 12.14). La versión correcta:

App.xaml.cs (versión final)

```

using NutriTrack.MAUI.Services;

namespace NutriTrack.MAUI
{
    public partial class App : Application
    {
        private readonly ISesionService _sesionService;

        public App(ISesionService sesionService)
        {
            InitializeComponent(); // 1) carga Colors.xaml y Styles.xaml
            UserAppTheme = AppTheme.Light; // tema claro fijo (como el prototipo)
            _sesionService = sesionService;
        }

        protected override Window CreateWindow(IActivationState? activationState)
        {
            // 2) recién acá se crea el Shell, cuando los colores ya existen
            return new Window(new AppShell(_sesionService));
        }
    }
}

```

El orden correcto

1. DI crea `SesionService` (no usa colores, sin problema)
2. App ejecuta `InitializeComponent` → se cargan `Colors.xaml` y `Styles.xaml` ✓
3. `CreateWindow` → `new AppShell(...)` → busca "Primary" → ya existe ✓

15.5 MauiProgram.cs final

MauiProgram.cs (versión final)

```
using Microsoft.Extensions.Logging;
using NutriTrack.MAUI.Services;
using NutriTrack.MAUI.Views;

namespace NutriTrack.MAUI
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureFonts(fonts =>
                {
                    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
                });

            // Sesión del usuario: Singleton porque tiene que ser UNA sola
            // instancia compartida por toda la app
            builder.Services.AddSingleton<ISesionService, SesionService>();

            // Handler que agrega el token a cada pedido
            builder.Services.AddTransient<AuthHeaderHandler>();

            // Cliente HTTP para hablar con NutriTrack.API
            // Se configura una sola vez acá y todos los servicios lo reutilizan
            builder.Services.AddHttpClient("NutriTrackApi", client =>
            {
                client.BaseAddress = new Uri(ApiConfig.BaseUrl);
                client.Timeout = TimeSpan.FromSeconds(15);
            })
                .AddHttpMessageHandler<AuthHeaderHandler>();

            // Navegación y pantallas
            builder.Services.AddTransient<LoginPage>();
            builder.Services.AddTransient<PanelPrincipalPage>();

#if DEBUG
            builder.Logging.AddDebug();
#endif

            return builder.Build();
        }
    }
}
```

- Las pantallas son **Transient**: cada vez que se entra, se crea limpia (sin errores ni campos viejos).
- El **AppShell** **no** se registra en DI: lo crea **App** en **CreateWindow**.

15.6 Chau robot

1. Eliminar **MainPage.xaml** (se borra junto con su **.xaml.cs**).
2. Eliminar **Resources/Images/dotnet_bot.png**.
3. Borrar del **.csproj** la línea **<MauiImage Update="Resources\Images\dotnet_bot.png" ... />** si existe.

Verificación final de la Fase 0

1. Al abrir: login provisorio con fondo gris claro, "NutriTrack" grande, sin barra arriba, botón verde. La ventana dice "NutriTrack".
2. "Entrar": panel con barra verde "Panel Principal" en blanco y **sin flecha de volver**.
3. "Cerrar sesión": vuelta al login.

Commit final

```
git add NutriTrack.MAUI
git commit -m "chore(maui): navegación con Shell, pantallas provisorias de login y panel, quitar plantilla"
git push
```

Resumen de commits de la rama

Commit	Mensaje
97bfcd9	paquetes MVVM/Http, targets Windows y Android, paleta NutriTrack y tema claro
55e5927	configurar HttpClient y URL base de la API por plataforma
bba9139	servicio de sesión con SecureStorage y quitar targets iOS/Mac
a7d2438	handler que agrega el token JWT a cada pedido
0e11f24	resultado de API y clase base de servicios con manejo de errores
36fa166	BaseViewModel con estado de carga y errores
49017bd	navegación con Shell, pantallas provisorias de login y panel, quitar plantilla

8. Estructura final: qué va en cada archivo

Árbol del proyecto al terminar la Fase 0

NutriTrack.MAUI/	
└─ App.xaml / App.xaml.cs	Arranque: carga recursos, fija tema claro, crea la ventana con el Shell
└─ AppShell.xaml / .xaml.cs	Mapa de navegación, rutas, sesión restaurada y sesión expirada
└─ MauiProgram.cs	Registro de servicios (DI) y configuración del HttpClient
└─ NutriTrack.MAUI.csproj	Plataformas, nombre visible, ícono, paquetes NuGet
└─ Models/	
└─ RolUsuario.cs	Enum de roles (copia exacta del enum del back)
└─ SesionUsuario.cs	Datos del usuario logueado (token, expiración, nombre, rol)
└─ ResultadoApi.cs	Resultado de cualquier pedido (éxito + datos / error + mensaje)
└─ Services/	
└─ ApiConfig.cs	URL base de la API según la plataforma
└─ ISesionService.cs	Contrato de la sesión
└─ SesionService.cs	Implementación: memoria + SecureStorage
└─ AuthHeaderHandler.cs	Agrega el token a cada pedido y detecta 401
└─ ApiServiceBase.cs	Clase base de servicios: GET/POST/PUT/DELETE + manejo de errores
└─ ViewModels/	
└─ BaseViewModel.cs	EstaOcupado, MensajeError, EjecutarAsync
└─ Views/	
└─ LoginPage.xaml / .xaml.cs	(provisoria; Fase 1)
└─ PanelPrincipalPage.xaml / .xaml.cs	(provisoria; Fase 2)
└─ Helpers/	
└─ Platforms/	(vacía por ahora)
└─ Android/	(../Resources/values/colors.xml: colores del sistema Android)
└─ Windows/	
└─ Resources/	
└─ Styles/Colors.xaml	Paleta de colores (con nombres)
└─ Styles/Styles.xaml	Reglas de estilo de la plantilla (usan la paleta)
└─ AppIcon/, Splash/, Fonts/, Images/	

8.1 Tabla de responsabilidades

Archivo	Responsabilidad	Depende de	Lo usan
MauiProgram.cs	Armar la app: registrar servicios y configurar el HttpClient	Todos los servicios y pantallas	MAUI al arrancar
App.xaml.cs	Cargar recursos, fijar tema, crear la ventana	ISesionService	MAUI
AppShell	Rutas, navegación, restaurar sesión, reaccionar a sesión expirada	ISesionService	App
ApiConfig	Dirección de la API por plataforma	—	MauiProgram
SesionService	Única fuente de verdad del usuario logueado	SecureStorage	Handler, Shell, App, (futuro) AuthService y ViewModels
AuthHeaderHandler	Token en cada pedido; 401 → sesión expirada	ISesionService	El HttpClient "NutriTrackApi"

Archivo	Responsabilidad	Depende de	Lo usan
<code>ApiServiceBase</code>	Hacer pedidos y convertir cualquier error en <code>ResultadoApi</code>	<code>IHttpClientFactory</code>	Todos los servicios que heredan
<code>ResultadoApi<T></code>	Transportar éxito/datos o error/mensaje	—	Servicios y ViewModels
<code>BaseViewModel</code>	Estado de carga, errores y protección contra doble envío	<code>CommunityToolkit.Mvvm</code>	Todos los ViewModels
<code>Colors.xaml</code>	Paleta con nombres	—	<code>Styles.xaml</code> y todas las Views
<code>Views/*.xaml</code>	Dibujar la pantalla	Su ViewModel (por Binding)	Shell

8.2 Convención de nombres

Tipo de archivo	Convención	Ejemplo
Pantalla	<code><Nombre>Page.xaml</code>	<code>AnimalesPage.xaml</code> , <code>FichaAnimalPage.xaml</code>
ViewModel	<code><Nombre>ViewModel.cs</code> (mismo nombre que la página)	<code>AnimalesViewModel.cs</code>
Servicio	<code>I<Nombre>Service.cs</code> + <code><Nombre>Service.cs</code>	<code>IAnimalService</code> , <code>AnimalService</code>
DTO de la API	Mismo nombre que en el back, terminando en <code>Dto</code> / <code>Request</code> / <code>Response</code>	<code>AnimalDto</code> , <code>LoginRequest</code> , <code>LoginResponse</code>
Ruta de navegación	camelCase, en castellano	"animales", "fichaAnimal"
Color	Según el uso, no el aspecto	<code>Peligro</code> , <code>TextoSecundario</code>
Namespace	Según la carpeta	<code>NutriTrack.MAUI.Views</code> , <code>NutriTrack.MAUI.Services</code>

9. Cómo se comunican los archivos

9.1 Secuencia de arranque de la app

Entender el orden de arranque explica el error del **Primary** (12.14) y es clave para no repetirlo:

Del ícono a la primera pantalla

```

MauiProgram.CreateMauiApp()
├─ registra servicios: SesionService (Singleton), AuthHeaderHandler (Transient),
│   HttpClient "NutriTrackApi" (+ handler), LoginPage y PanelPrincipalPage (Transient)
└─ builder.Build()
    │
    ▼
    MAUI pide un App a DI
    └─ DI ve que App necesita ISesionService → crea SesionService → new App(sesion)
        │
        ▼
        App() — InitializeComponent() → carga App.xaml → Colors.xaml + Styles.xaml (recursos listos)
              — UserAppTheme = Light
            │
            ▼
            App.CreateWindow() → new AppShell(sesion)
            └─ AppShell() — InitializeComponent() → usa {StaticResource Primary} ✓ ya existe
                          — se suscribe a SesionExpirada
                          — Loaded += OnLoaded
                │
                ▼
                Se muestra la primera ShellContent: "login" → DI crea LoginPage
                │
                ▼
                OnLoaded → RestaurarSesionAsync()
                └─ hay sesión guardada y vigente → GoToAsync("//panel")
                  └─ no hay → queda en el login
  
```

9.2 Recorrido de un pedido (ejemplo: Ingresar)

El pedido baja por las capas, cruza la red y la respuesta sube



La Fase 0 dejó listos los pasos ④ y el manejo de la respuesta en ③. La Fase 1 agrega ①, ② y el **AuthService**. En la prueba del 200 OK el botón saltó directo de ① a ④: funcionó, pero hacerlo en todas las pantallas mezclaría código HTTP con la interfaz.

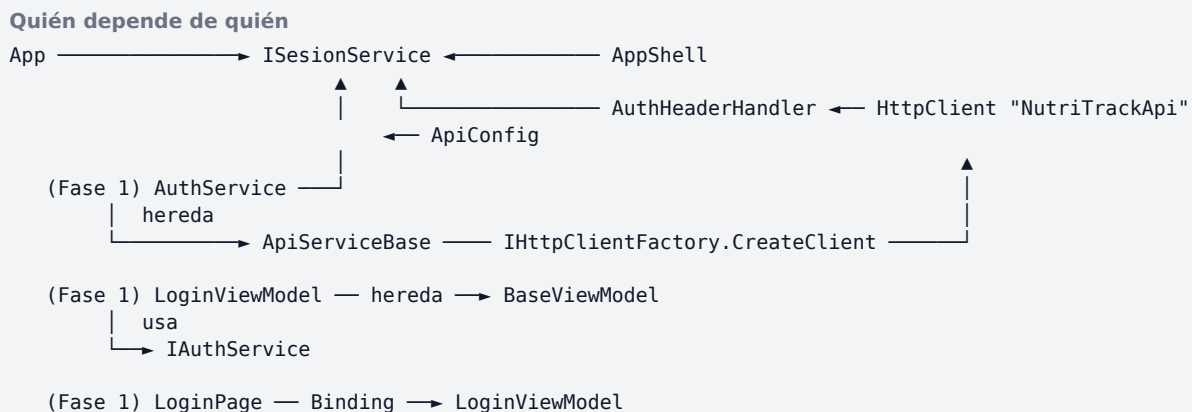
9.3 Recorrido cuando el token vence

```

AnimalesViewModel → AnimalService.GetAsync(...) → HttpClient
→ AuthHeaderHandler agrega "Bearer <token viejo>"
→ API responde 401
→ AuthHeaderHandler: 401 && había sesión → SessionService.NotificarSesionExpirada()
→ CerrarSesion() (borra memoria y SecureStorage)
→ dispara el evento SesionExpirada
→ AppShell.OnSesionExpirada → (hilo principal) alerta "Sesión expirada" →
GoToAsync("//login")
→ ApiServiceBase convierte el 401 en ResultadoApi.Error("Tu sesión expiró...")
→ el ViewModel recibe el resultado sin haber escrito ni un try/catch

```

9.4 Mapa de dependencias



9.5 Ciclos de vida registrados

Registro	Lifetime	Por qué
<code>ISessionService</code> → <code>SesionService</code>	Singleton	Una única sesión para toda la app
<code>AuthHeaderHandler</code>	Transient	Requisito de <code>AddHttpMessageHandler</code>
<code>HttpClient</code> "NutriTrackApi"	Lo administra <code>IHttpClientFactory</code>	Reutiliza conexiones de forma segura
<code>LoginPage</code> , <code>PanelPrincipalPage</code>	Transient	Cada visita a la pantalla empieza limpia
(Fase 1) ViewModels	Transient	Igual que su página
(Fase 1) Servicios de API	Transient o Singleton	Sin estado propio: cualquiera funciona; Transient es lo más simple
<code>AppShell</code>	No se registra	Lo crea App en <code>CreateWindow</code> (después de cargar los recursos)

10. Comunicación front ↔ back

10.1 Contrato del login

```
POST /api/login
Content-Type: application/json

{ "nombreUsuario": "admin", "contrasenia": "*****" }

200 OK → { "token": "eyJ...", "expiracion": "2026-10-01T02:30:00Z", "nombreUsuario": "admin",
  "rol": "Administrador" }
401 → mensaje genérico de credenciales inválidas (mismo para usuario inexistente o contraseña
mal)
400 → campos vacíos o demasiado largos ([Required], [MaxLength] del LoginDTO)
```

10.2 Reglas que front y back tienen que compartir

Regla	En el back	En el front
Nombres en camelCase	Comportamiento por defecto de ASP.NET	JsonSerializerDefaults.Web en OpcionesJson
Enums como texto	JsonStringEnumConverter en Program.cs	El mismo converter en OpcionesJson
Nombres de los roles	NutriTrack.Core...RolUsuario	Models/RolUsuario.cs (copia exacta)
Fechas del token en UTC	TokenService (8 horas)	DateTime.UtcNow en EstaVigente
Token en cada pedido	Fallback policy: todo exige autenticación	AuthHeaderHandler
Mensajes de error	BadRequest("...") con texto	ApiServiceBase los muestra tal cual

10.3 Cómo levantar todo para desarrollar

1. Base de datos levantada en Docker.
2. Configuración del back: `Jwt:Issuer` y `Jwt:Audience` en `appsettings.json`, `Jwt:Key` en user secrets (instrucciones en el README).
3. API en una terminal: `dotnet run --project NutriTrack.API --launch-profile http`.
4. Visual Studio: proyecto de inicio `NutriTrack.MAUI`, ▶ con Windows Machine.
5. Para Android (más adelante): emulador, y la URL `10.0.2.2` ya está resuelta en `ApiConfig`.

Atención

Para probar contra Android con HTTP (sin S) puede hacer falta habilitar el tráfico sin cifrar en modo Debug, porque Android lo bloquea por defecto. Se resuelve cuando se pruebe en el emulador.

10.4 Probar la API sin el front

Para saber si un problema es del front o del back, conviene probar el mismo pedido en Scalar:

1. `POST /api/login` con un usuario real (contraseña hasheada con Argon2).
2. Copiar el `token` de la respuesta.
3. En Scalar: **Authentication** → **Bearer** y pegarlo.
4. Si en Scalar funciona y en la app no, el problema es del front. Si falla en los dos, es del back.

Tip

Con la API corriendo en una terminal se ven los logs en vivo. Si al tocar un botón no aparece nada en la terminal, el pedido nunca llegó a la API.

11. Cómo agregar una pantalla nueva

Esta es la convención del equipo. Se ilustra con un ejemplo: una pantalla que lista los ingredientes (GET [api/Ingrediente](#), el mismo endpoint de la prueba del 200 OK).

Nota

El ejemplo es ilustrativo. Las propiedades del DTO tienen que coincidir con lo que devuelve realmente el endpoint: verifícalo primero en Scalar.

Paso A: el modelo (DTO)

Models/IngredienteDto.cs

```
namespace NutriTrack.MAUI.Models
{
    public class IngredienteDto
    {
        public int IdIngrediente { get; set; }
        public string Nombre { get; set; } = string.Empty;
        public string? Descripcion { get; set; }
    }
}
```

Paso B: el servicio

Services/IngredienteService.cs

```
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    public interface IIngredienteService
    {
        Task<ResultadoApi<List<IngredienteDto>>> ObtenerTodosAsync();
    }

    public class IngredienteService : ApiServiceBase, IIngredienteService
    {
        public IngredienteService(IHttpClientFactory httpClientFactory) : base(httpClientFactory) {

        }

        public Task<ResultadoApi<List<IngredienteDto>>> ObtenerTodosAsync() =>
            GetAsync<List<IngredienteDto>>("api/Ingrediente");
    }
}
```

No hay manejo de errores ni de token: lo heredan de [ApiServiceBase](#) y del handler.

Paso C: el ViewModel

ViewModels/IngredientesViewModel.cs

```
using System.Collections.ObjectModel;
using CommunityToolkit.Mvvm.Input;
using NutriTrack.MAUI.Models;
using NutriTrack.MAUI.Services;

namespace NutriTrack.MAUI.ViewModels
{
    public partial class IngredientesViewModel : BaseViewModel
    {
        private readonly IIngredienteService _ingredienteService;

        public ObservableCollection<IngredienteDto> Ingredientes { get; } = new();

        public IngredientesViewModel(IIngredienteService ingredienteService)
        {
            _ingredienteService = ingredienteService;
        }

        [RelayCommand]
        private Task Cargar() => EjecutarAsync(async () =>
        {
            var resultado = await _ingredienteService.ObtenerTodosAsync();
            if (!resultado.Exito)
            {
                MensajeError = resultado.MensajeError;
                return;
            }

            Ingredientes.Clear();
            foreach (var i in resultado.Datos ?? new())
                Ingredientes.Add(i);
        });
    }
}
```

ObservableCollection es una lista que avisa a la pantalla cuando se agregan o quitan elementos (la versión "observable" de **List**).

Paso D: la View

Views/IngredientesPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:vm="clr-namespace:NutriTrack.MAUI.ViewModels"
    xmlns:m="clr-namespace:NutriTrack.MAUI.Models"
    x:Class="NutriTrack.MAUI.Views.IngredientesPage"
    x:DataType="vm:IngredientesViewModel"
    Title="Ingredientes"
    BackgroundColor="{StaticResource Fondo}">

    <Grid RowDefinitions="Auto,Auto,*" Padding="16" RowSpacing="12">
        <ActivityIndicator IsRunning="{Binding EstaOcupado}" IsVisible="{Binding EstaOcupado}" />
        <Label Grid.Row="1" Text="{Binding MensajeError}" IsVisible="{Binding HayError}"
            TextColor="{StaticResource Peligro}" />
        <CollectionView Grid.Row="2" ItemsSource="{Binding Ingredientes}">
            <CollectionView.ItemTemplate>
                <DataTemplate x:DataType="m:IngredienteDto">
                    <Label Text="{Binding Nombre}" Padding="0,8"
                        TextColor="{StaticResource TextoPrincipal}" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>
```

Views/IngredientesPage.xaml.cs

```
using NutriTrack.MAUI.ViewModels;

namespace NutriTrack.MAUI.Views
{
    public partial class IngredientesPage : ContentPage
    {
        private readonly IngredientesViewModel _vm;

        public IngredientesPage(IngredientesViewModel vm)
        {
            InitializeComponent();
            BindingContext = _vm = vm; // conecta la View con su ViewModel
        }

        protected override async void OnAppearing()
        {
            base.OnAppearing();
            await _vm.CargarCommand.ExecuteAsync(null);
        }
    }
}
```

- **BindingContext = vm** es lo que conecta la pantalla con el ViewModel: a partir de ahí todos los **{Binding ...}** buscan en el ViewModel.
- **x:DataType** activa los *compiled bindings*: el compilador avisa si una propiedad está mal escrita.
- **OnAppearing** carga los datos cada vez que se muestra la pantalla.

Paso E: registrar y navegar

```
// MauiProgram.cs
builder.Services.AddTransient<IIngredienteService, IngredienteService>();
builder.Services.AddTransient<IngredientesViewModel>();
builder.Services.AddTransient<IngredientesPage>();

// AppShell.xaml.cs (constructor)
Routing.RegisterRoute("ingredientes", typeof(IngredientesPage));

// Desde otra pantalla (navegación relativa: aparece la flecha de volver)
await Shell.Current.GoToAsync("ingredientes");
```

Checklist

#	Verificar
1	El DTO coincide con lo que devuelve la API (probado en Scalar).
2	El servicio hereda de <code>ApiServiceBase</code> y tiene su interfaz.
3	El ViewModel hereda de <code>BaseViewModel</code> y envuelve las llamadas en <code>EjecutarAsync</code> .
4	La View no tiene lógica: solo Bindings. Usa colores de la paleta (nada de hex sueltos).
5	Servicio, ViewModel y página registrados en <code>MauiProgram.cs</code> .
6	Ruta registrada con <code>Routing.RegisterRoute</code> (si se apila) y navegación sin <code>//</code> .
7	Si el endpoint no existe en la API, se agrega en el back (acuerdo del equipo).
8	Compila sin errores y se probó: datos, error de API y sin conexión (API apagada).

12. Catálogo de errores y cómo resolverlos

Todos estos errores aparecieron realmente durante la Fase 0. Están en el orden en que surgieron.

12.0 Reglas generales para cualquier error

1. **Leé el mensaje completo**, en especial la línea en negrita y las columnas **Proyecto** y **Archivo** de la Lista de errores. Muchas veces el error no es del proyecto donde estás trabajando.
2. **Arreglá el primer error de la lista y recompilá**. Muchos errores son consecuencia de otro (errores en cascada).
3. **Rojo = error, amarillo = advertencia**. Las advertencias no impiden compilar.
4. **Distinguí compilación de ejecución**. Si compila pero explota al abrir, el problema suele estar en el XAML o en el orden en que se crean las cosas.
5. **Si un error "no tiene sentido" o no desaparece**: Compilar → Limpiar solución, y después Recompilar.
6. **El código generado no se toca**. Si Visual Studio muestra archivos con `xamlServiceProvider` o `__root`, es código que MAUI genera a partir de tus XAML. El problema real está en tu XAML.

12.1 a 12.4: git

12.1 La terminal muestra (END) y líneas con ~

Síntoma	Después de <code>git diff</code> la salida se repite y aparecen ~ y (END).
Causa	Git abrió el resultado en un visor de texto (<i>pager</i>) y seguís adentro.
Solución	Apretar <code>q</code> .
Cómo evitarlo	Usar <code>git --no-pager diff ...</code> .

12.2 fatal: not a git repository

Síntoma	<code>fatal: not a git repository (or any of the parent directories): .git</code>
Causa	La terminal está en <code>C:\Users\Daniela\source\repos</code> (la carpeta que contiene los proyectos) y no en el repo. Git reconoce un repo por la carpeta oculta <code>.git</code> .
Solución	<code>cd NutriTrack</code> y repetir el comando.
Cómo evitarlo	Antes de cualquier comando de git, mirar que la ruta termine en <code>\NutriTrack></code> . La Developer PowerShell de Visual Studio suele abrirse ya en la carpeta de la solución.

12.3 Un cambio "me sigue" de rama en rama (M en git status)

Síntoma	Al cambiar de rama aparece <code>M NutriTrack.API/DTOs/UsuarioResponseDTO.cs</code> .
Causa	Los cambios sin commitar no pertenecen a ninguna rama: viven en tu carpeta y te acompañan.

Solución	Ver qué es: <code>git --no-pager diff <archivo></code> . Si es un arreglo válido de otra cosa, llevarlo a su propia rama: <code>git checkout main → git checkout -b fix/... → git add <archivo> → git commit -m "..."</code> → <code>git push -u origin fix/...</code> → volver a tu rama. Si no sirve: <code>git restore <archivo></code> .
Cómo evitarlo	Comitear con <code>git add <carpeta o archivo></code> y no con <code>git add ..</code> Revisar <code>git status</code> antes de cada commit.

12.4 Archivo Untracked y la API de main que no compila

Síntoma	Untracked files: NutriTrack.Infraestructure/Repositories/Reportefechasimportantesrepository.cs. Además, la API en main no compilaba.
Causa	El commit del CU18 subió el controller, los DTOs y el PDF, pero no el repository (se olvidó en el <code>git add</code>). <code>Program.cs</code> y el controller usaban una clase que no existía en GitHub. Lo resolvió el PR #75. Aparte, <code>UsuarioResponseDTO.cs</code> usaba <code>RoUsuario</code> con <code>using NutriTrack.Core.Entities</code> ; cuando el enum está en <code>NutriTrack.Core.Entities.Enums</code> (CS0246). Se arregló en la rama <code>fix/using-rousuario-usuario-response</code> .
Solución	Para seguir trabajando: corregir el <code>using</code> localmente sin commitarlo en la rama del front. Cuando se mergee el fix: <code>git restore <archivo></code> y <code>git merge main</code> . Si un archivo untracked choca con uno que llega por <code>git pull</code> ("untracked working tree files would be overwritten"): borrar la copia local y repetir el pull.
Cómo evitarlo	Revisar <code>git status</code> antes de cada commit: ahí aparecen los archivos que quedaron afuera. En C#, el archivo se llama igual que la clase.

12.5 a 12.7: ejecutar y compilar

12.5 No se puede iniciar el programa ... NutriTrack.API.exe

Síntoma	Ventana: "El sistema no puede encontrar el archivo especificado" al iniciar la API.
Causa	La API no compiló (por el error del <code>using</code>), así que no se generó el <code>.exe</code> .
Solución	Abrir Ver → Lista de errores , arreglar el error de compilación y volver a ejecutar.
Cómo evitarlo	Cuando algo no arranca, lo primero es mirar la Lista de errores.

12.6 An element with the same key ... SelfHostWebServer

Síntoma	Al iniciar con "Varios proyectos de inicio": An element with the same key but a different value already exists. Key: 'Microsoft.WebTools.ProjectSystem.WebServer.SelfHostWebServer'.
Causa	Bug interno de Visual Studio al levantar dos proyectos a la vez. No es del código.
Solución	Opción A: cerrar y reabrir Visual Studio. Opción B (recomendada): volver a Proyecto de inicio único (NutriTrack.MAUI) y correr la API en una terminal con <code>dotnet run --project NutriTrack.API --launch-profile http</code> .
Cómo evitarlo	Trabajar siempre con la API en una terminal aparte.

12.7 MSB3027 / MSB3021: no se pudo copiar ... bloqueado por NutriTrack.API

Síntoma	No se pudo copiar ... NutriTrack.Core.dll ... El archivo se ha bloqueado por: "NutriTrack.API (22332)".
Causa	Se recompiló toda la solución (incluida la API) mientras la API corría en la terminal. Windows no deja reemplazar archivos en uso.
Solución	Apagar la API con Ctrl + C en su terminal antes de recompilar la solución, o recompilar solo el MAUI : clic derecho en NutriTrack.MAUI → Recompilar. El ► compila solo el proyecto de inicio, así que no choca.
Cómo evitarlo	Mirar la columna Proyecto: si dice NutriTrack.API , no es un error del front. En la Lista de errores se puede filtrar por Proyecto actual .

12.8 a 12.9: colores y recursos

12.8 XamlParseException: StaticResource not found for key 'White'

Síntoma	Compila, pero al abrir la app aparece una excepción en código generado. Position 7:34.
Causa	Al reemplazar los colores violetas se borraron también White , Black , los grises y los Brush de Colors.xaml . La línea 7 de Styles.xaml (el ActivityIndicator) pide White y no existe.
Solución	Detener la app y cerrar la pestaña de código generado. Reemplazar todo Colors.xaml por la versión completa (Paso 7). Si sigue igual: verificar que el archivo esté guardado (sin * en la pestaña) y hacer Limpiar solución + Recompilar.
Cómo evitarlo	Respetar el comentario "NO BORRAR". Los StaticResource no se verifican al compilar.

12.9 Colors.xaml y colors.xml confundidos

Síntoma	El error 12.8 seguía después de "arreglarlo".
Causa	La paleta completa se había pegado en Platforms/Android/Resources/values/colors.xml (el de Android) en lugar de Resources/Styles/Colors.xaml . Quedaron cruzados.
Solución	Poner la paleta completa en Colors.xaml y los tres colores en el de Android (Paso 7). Si no hay Ctrl + Z disponible: git restore <archivo> recupera la versión del último commit.
Cómo evitarlo	Colors.xaml : <Color x:Key> . Android: <resources> , <color name> . El archivo de Android roto habría hecho fallar la app en Android aunque Windows compilara.

12.10 a 12.12: compilación del proyecto MAUI

12.10 CS5001: el programa no contiene ningún método Main

Síntoma	Al recompilar la solución, error en <code>NutriTrack.MAUI (net10.0-ios...)</code> .
Causa	Se borraron las carpetas <code>Platforms/iOS</code> y <code>MacCatalyst</code> pero no la línea del <code>.csproj</code> con <code>net10.0-ios;net10.0-maccatalyst</code> . El proyecto seguía compilando para iOS sin su punto de entrada. Con ► en Windows no aparecía porque solo compila Windows.
Solución	Borrar esa línea del <code>.csproj</code> (Paso 5).
Cómo evitarlo	Al cambiar plataformas, revisar el <code>.csproj</code> en GitHub o en Dependencias.

12.11 "No se encontraron problemas" pero no sé si compiló

Síntoma	Abajo del editor dice "No se encontraron problemas" y no aparece la ventana Salida.
Causa	Ese indicador se refiere solo al archivo abierto y a veces tarda en actualizarse. La ventana Salida estaba cerrada.
Solución	Abrir Ver → Salida (Ctrl + Alt + O) y buscar 1 correcto, 0 incorrectos . O Ver → Lista de errores : debe decir 0 Errores .
Cómo evitarlo	Verificar la compilación siempre en Salida o en la Lista de errores.

12.12 Texto suelto en el código (JG en lugar de #if DEBUG)

Síntoma	En <code>MauiProgram.cs</code> apareció <code>JG</code> donde iba <code>#if DEBUG</code> .
Causa	Una tecla o un pegado accidental.
Solución	Restaurar <code>#if DEBUG</code> y verificar que exista su <code>#endif</code> debajo de <code>builder.Logging.AddDebug();</code> .
Cómo evitarlo	Revisar el archivo después de pegar.

12.13 a 12.14: XAML y orden de ejecución

12.13 Data at the root level is invalid + CS0103 InitializeComponent

Síntoma	Dos errores: <code>An error occurred while parsing Xaml: Data at the root level is invalid</code> en <code>PanelPrincipalPage.xaml</code> (línea 16) y <code>CS0103: El nombre 'InitializeComponent' no existe en el .xaml.cs</code> .
Causa	Quedó contenido después del <code></ContentPage></code> final (lo que traía la plantilla o un pegado doble). Un XAML tiene una sola etiqueta raíz y después no puede haber nada. El CS0103 es un error en cascada : como el XAML no se pudo leer, MAUI no generó <code>InitializeComponent</code> .
Solución	Ctrl + A en el XAML, pegar el contenido correcto, verificar que la última línea sea <code></ContentPage></code> y recompilar.
Cómo evitarlo	Al reemplazar un archivo: siempre Ctrl + A y después pegar. Arreglar el primer error de la lista.

12.14.XamlParseException: StaticResource not found for key 'Primary' (orden de ejecución)

Síntoma	Compila, pero al abrir explota en el código generado de <code>AppShell</code> (<code>Route = "login"</code>).
Causa	En la primera versión, <code>App</code> recibía el <code>AppShell</code> por constructor. Para pasárselo, DI creaba el <code>Shell</code> antes que <code>App</code> , y el <code>Shell</code> buscaba <code>Primary</code> cuando <code>Colors.xaml</code> todavía no se había cargado (eso pasa en el <code>InitializeComponent</code> de <code>App</code>). Cada archivo por separado estaba bien: el problema era cuándo se ejecutaba cada cosa.
Solución	<code>App</code> recibe <code>ISessionService</code> y crea el <code>Shell</code> en <code>CreateWindow</code> , después de cargar los recursos. Se borra <code>AddSingleton<AppShell>()</code> (ver Paso 15.4).
Cómo evitarlo	Pensar en el orden de ejecución. Todo lo que use recursos de la app tiene que crearse después del <code>InitializeComponent</code> de <code>App</code> .

12.15 Advertencias que se pueden ignorar

Código	Mensaje	Qué hacer
CS8618	La propiedad X que no acepta NULL debe contener un valor... (en <code>NutriTrack.Core</code>)	Son del back y ya existían. No impiden compilar.
CA1416	Se puede acceder a este sitio de llamada en todas las plataformas... solo se admite en Windows 10.0.17763.0	Avisos de compatibilidad de plataforma. El <code>.csproj</code> está bien configurado; pueden aparecer mientras hay otro error. No impiden ejecutar.

12.16 Dudas de Visual Studio

Duda	Respuesta
No encuentro <code>MainPage.xaml.cs</code>	Está agrupado bajo <code>MainPage.xaml</code> : tocar la flechita a la izquierda, o F7.
La ventana de "Nuevo elemento" no tiene lista	Es la versión compacta: tocar Mostrar todas las plantillas y usar el buscador.
Dependencias muestra cuatro carpetas	En MAUI las dependencias se ven por plataforma; los paquetes están en cada una, en Paquetes .
La barra de íconos arriba de la app	Herramienta de depuración de Visual Studio; solo aparece al ejecutar desde el IDE.
La app se ve oscura	Copia el tema de Windows. Se fijó el tema claro con <code>UserAppTheme = AppTheme.Light</code> .

13. Preguntas que surgieron en el proceso

Estas son las preguntas reales que aparecieron mientras se armaba la Fase 0, con la respuesta resumida. Probablemente sean las mismas que le surjan a cualquier integrante nuevo.

¿Qué le pregunto al compañero que configuró el JWT y qué plan le mando?

Si el endpoint de login estaba hecho y sin subir; si el login era por nombre de usuario o correo; quién regeneraba los usuarios de prueba (contraseñas falsas, roles mal escritos); y que la API en Docker no tenía la clave. Se le mandó la división en fases y la propuesta de que la base del front fuera primero en un PR chico. Se preparó también una versión menos técnica del mensaje.

¿Por qué la clave tiene que ir en el compose?

Porque los user secrets viven en la PC de cada uno y el contenedor no los ve. El compose pasa configuración por variables de entorno (`Jwt__Key: ${JWT_KEY}`), con el valor en el `.env` que no se sube. Solo importa si la API corre dentro de Docker (sección 3.4).

¿No puedo empezar el login todavía?

Sí se podía: lo único trabado era recibir el token. La pantalla, el ViewModel, la sesión y la navegación no dependen de eso, y se podía usar un `AuthServiceFalso` detrás de una interfaz. Al final el login real llegó antes.

¿Tengo que instalar algo? ¿Hago una rama y empiezo?

Solo verificar el workload de MAUI (ya estaba). Rama nueva desde main actualizado, con prefijo `chore/`. Ni emulador ni paquetes antes de tiempo.

No sé si instalé los paquetes.

`git status`: si aparece el `.csproj` modificado, sí. O buscar `<PackageReference` en el `.csproj`. En el peor caso se instalan de nuevo.

¿Qué onda el cambio del nombre?

`ApplicationTitle` es el nombre visible para el usuario (barra de la ventana, nombre bajo el ícono). No cambia proyectos, namespaces ni carpetas.

¿Hasta acá solo se configuró?

Sí: cimientos. Primero las bases y las cañerías; después las paredes. Si se empieza por la pantalla, a mitad de camino hay que frenar para armar todo esto igual.

¿Subo el PR ya o termino lo que falta?

Terminar la Fase 0: sin token, errores, `BaseViewModel` y navegación, los compañeros no pueden hacer pantallas que funcionen. La única razón para subirlo antes sería que alguien necesite solo la estructura para diseñar XAML sin conectar.

¿Cómo se conectan las pantallas? Se comunicó el front con el back, pero ¿cómo?

View → (Binding) → ViewModel → Service → HttpClient (+ handler) → API → Controller → Repository → base, y la respuesta sube por el mismo camino (capítulo 9). La prueba del 200 OK fue un atajo que saltó directo de la pantalla al HttpClient.

La navegación absoluta, ¿es solo para el login?

Sí: solo para entrar y salir de la sesión. Todo lo demás es relativa y se registra con `Routing.RegisterRoute` en `AppShell.xaml.cs`.

¿Le falta información al PR?

Estaba bien; se recomendó sumar capturas, qué quedó sin probar y el aviso sobre la API de main.

¿Este PR para qué es, si no hice nada del login?

Es la base para que todos trabajen en paralelo. Además contiene la parte invisible del login (conexión, sesión, token, errores, carga y navegación); falta solo pantalla, ViewModel y AuthService.

GitHub dice "recent pushes 3 hours ago" y recién hice push.

Es solo el texto del cartel, que no se actualiza al instante. Se verificó que el último commit estaba en GitHub. El PR toma todo lo que hay en la rama.

Me siento tonta aprendiendo esto.

Se estaba aprendiendo un framework nuevo, un patrón de arquitectura nuevo y git a la vez, y se resolvieron errores reales (en cascada, de orden de ejecución, de archivos bloqueados) que le pasan a cualquiera que empieza con MAUI. Sentirse perdida al principio es parte de aprender algo de verdad.

14. Decisiones de diseño (para la tesis)

Decisión	Alternativas	Justificación	Relación
MVVM con CommunityToolkit.Mvvm	Code-behind	Separación de responsabilidades, bajo acoplamiento, menos código repetido (generadores)	RNF5
Solo Windows y Android	Mantener las 4 plataformas	Alcance del proyecto; compilación más rápida; iOS requiere Mac	RNF2
Tema claro fijo	Tema claro + oscuro	Consistencia con el prototipo validado	RNF1
Paleta con nombres semánticos	Colores sueltos en cada pantalla	Un cambio de color en un solo lugar	RNF5
<code>AddHttpClient</code> con cliente con nombre	<code>new HttpClient()</code> en cada pantalla	Reutilización segura de conexiones; configuración única	RNF3, RNF5
Timeout de 15 s	Sin timeout / el de .NET (100 s)	Evitar que la app quede colgada en redes lentas	RNF3, RNF1
<code>SecureStorage</code> + memoria	Preferences / solo memoria	Token cifrado y sesión recordada	RNF4
Recordar sesión	Login siempre	Mejor experiencia para quien usa la app todo el día	RNF1
<code>AuthHeaderHandler</code>	Agregar el token a mano en cada servicio	Un solo lugar; imposible olvidarse	RNF4, RNF5
Patrón Result	Excepciones	Los errores esperables son resultados; la app no crashea	RNF1, RNF7
No mostrar detalle de errores 500	Mostrar el mensaje del servidor	Seguridad y claridad para el usuario	RNF4
<code>BaseViewModel</code> + <code>EjecutarAsync</code>	Cada <code>ViewModel</code> resuelve carga y errores	Consistencia y protección contra doble envío	RNF1, RNF7
Enum de roles copiado en el front	Referenciar <code>NutriTrack.Core</code>	Front desacoplado de las entidades del back	RNF5
Navegación absoluta solo para la sesión	Todo relativo	El login no queda en el historial	RNF4
La sesión avisa por evento	El servicio navega	Los servicios no dependen de la interfaz	RNF5
API en terminal aparte	Varios proyectos de inicio	Estabilidad (bug de VS), independencia, logs visibles	—

Frases listas para el documento

Para la tesis

"Se adoptó el patrón MVVM para separar la lógica de presentación de la interfaz gráfica, favoreciendo el bajo acoplamiento y la mantenibilidad (RNF5)."

Para la tesis

"Se restringieron los target frameworks a Windows y Android conforme al alcance definido (RNF2, Portabilidad)."

Para la tesis

"Se fijó el tema claro para garantizar consistencia visual con el prototipo validado (RNF1, Usabilidad)."

Para la tesis

"Se validó la comunicación cliente-servidor mediante una petición GET a api/Ingrediente desde la aplicación MAUI, obteniendo respuesta HTTP 200."

Para la tesis

"El token de acceso se persiste cifrado mediante SecureStorage (RNF4, Seguridad)."

Para la tesis

"La incorporación del token JWT a cada petición se centraliza en un DelegatingHandler, análogo del lado cliente al middleware de autenticación del servidor."

Para la tesis

"Los errores de comunicación se modelan mediante el patrón Result, de modo que las situaciones esperables (credenciales inválidas, falta de conectividad) no interrumpen la ejecución de la aplicación."

Para la tesis

"Ocultar opciones en la interfaz según el rol mejora la experiencia de usuario, pero no constituye un mecanismo de seguridad; el control de acceso se aplica en el servidor."

15. Pendientes y próximos pasos

15.1 Pendientes del equipo

Pendiente	Por qué importa
Mergear <code>fix/using-rolusuario-usuario-response</code>	Sin eso la API de main no compila para nadie.
Administrador inicial con hash Argon2 real en el SQL	Sin él, en una base nueva nadie puede loguearse (huevo y gallina).
Regenerar usuarios de prueba (nombre_usuario, roles con el nombre del enum, hashes reales)	Los actuales no sirven para el login.
Actualizar <code>backup0k.sql</code> al esquema actual	Le faltan columnas de usuario.
<code>Jwt_Key</code> en el docker-compose y <code>JWT_KEY</code> en <code>.env.example</code>	Para el modo "todo en Docker".
Restricción por rol en los controllers	Fuera de alcance del PR del login; es la seguridad real por rol.
Endpoints de listado (animales, rodeos, planes) y resumen del panel	Los necesitan las fases 2 a 5.
Agregar <code>nombre</code> a la respuesta del login (opcional)	Para mostrar "Hola, Daniela" en el panel.

15.2 Pendiente de probar (depende de la Fase 1)

- Guardar y restaurar la sesión al reabrir la app.
- Token agregado a los pedidos y vuelta al login ante un 401.
- Mensajes de `ApiServiceBase` contra respuestas reales.
- La app en el emulador de Android (URL `10.0.2.2` y tráfico HTTP en Debug).

15.3 Fase 1: el login real

Rama nueva `feature/login`. Gracias a la Fase 0, son pocas piezas que se enchufan a lo existente:

Archivo	Qué hace
<code>Models/LoginRequest.cs</code>	<code>NombreUsuario</code> , <code>Contrasenia</code>
<code>Models/LoginResponse.cs</code>	<code>Token</code> , <code>Expiracion</code> , <code>NombreUsuario</code> , <code>Rol</code>
<code>Services/IAuthService.cs</code> + <code>AuthService.cs</code>	Hereda de <code>ApiServiceBase</code> ; <code>PostAsync<LoginResponse>("api/login", ...)</code> ; si sale bien guarda la sesión con <code>SesionService</code> ; si es 401 devuelve el mensaje de credenciales.
<code>ViewModels/LoginViewModel.cs</code>	Hereda de <code>BaseViewModel</code> ; propiedades <code>NombreUsuario</code> y <code>Contrasenia</code> ; comando <code>Ingresa</code> con <code>EjecutarAsync</code> ; navega con <code>"//panel"</code> .
<code>Views/LoginPage.xaml</code>	Diseño del prototipo: logo, campo usuario, campo contraseña (<code>IsPassword="True"</code>), botón "Ingresa", indicador de carga y mensaje de error.

Archivo	Qué hace
Panel provisorio	"Cerrar sesión" pasa a llamar a <code>SesionService.CerrarSesion()</code> antes de <code>///<code>login</code></code> ".

Adelanto del LoginViewModel

```
public partial class LoginViewModel : BaseViewModel
{
    [RelayCommand]
    private async Task Ingresar() => await EjecutarAsync(async () =>
    {
        var resultado = await _authService.IniciarSesionAsync(NombreUsuario, Contraseña);
        if (!resultado.Exito)
        {
            MensajeError = resultado.MensajeError; // viene de ApiServiceBase
            return;
        }
        await Shell.Current.GoToAsync("///panel");
    });
}
```

16. Comandos y atajos útiles

Git

Comando	Para qué
<code>git status</code>	Ver qué cambió y en qué rama estás
<code>git checkout main / git pull origin main</code>	Ir a main y traer lo último
<code>git checkout -b <rama></code>	Crear una rama y cambiarse a ella
<code>git add NutriTrack.MAUI</code>	Agregar solo la carpeta del front
<code>git commit -m "tipo(alcance): mensaje"</code>	Guardar un punto estable
<code>git push -u origin <rama> / git push</code>	Subir la rama (primera vez / siguientes)
<code>git restore <archivo></code>	Descartar cambios sin commitear
<code>git --no-pager diff <archivo></code>	Ver cambios sin abrir el visor
<code>git diff --stat</code>	Resumen de archivos cambiados
<code>git branch / git branch -d <rama></code>	Listar / borrar ramas locales
<code>git merge main</code>	Traer main a tu rama
q	Salir del visor de git

.NET y la API

Comando	Para qué
<code>dotnet workload list</code>	Ver qué workloads hay instalados
<code>dotnet run --project NutriTrack.API --launch-profile http</code>	Levantar la API en <code>http://localhost:5068</code>
Ctrl + C (en la terminal de la API)	Apagar la API
<code>dotnet build NutriTrack.API</code>	Compilar solo la API para verificar errores

Visual Studio

Acción	Cómo
Elegir proyecto de inicio	Clic derecho en el proyecto → Establecer como proyecto de inicio
Ejecutar en Windows	Desplegable del ► → Windows Machine → ►
Detener la app	Cuadrado rojo
Recompilar solo el MAUI	Clic derecho en <code>NutriTrack.MAUI</code> → Recompilar
Limpiar y recompilar todo	Compilar → Limpiar solución → Recompilar solución (con la API apagada)
Lista de errores	Ver → Lista de errores (filtrable por Proyecto actual)
Salida de la compilación	Ver → Salida (Ctrl + Alt + O)
Ir del XAML al code-behind	F7

Acción	Cómo
Guardar todo	Ctrl + Shift + S
Reemplazar un archivo entero	Ctrl + A y pegar
Paquetes NuGet	Clic derecho en el proyecto → Administrar paquetes NuGet...
Nueva pantalla	Clic derecho en Views → Agregar → Nuevo elemento → .NET MAUI ContentPage (XAML)

17. Glosario

Término	Definición
API	Programa del back que recibe pedidos HTTP y devuelve datos (NutriTrack.API).
AppThemeBinding	Elige un valor según el tema claro u oscuro del sistema.
async / await	Forma de esperar operaciones lentas (como HTTP) sin congelar la pantalla.
Bearer	Esquema del encabezado Authorization que lleva el token JWT.
Binding	Enlace entre una propiedad del ViewModel y un control de la View.
Code-behind	El archivo .xaml.cs de una pantalla.
Commit	Foto guardada de los archivos en git.
DelegatingHandler	Pieza que intercepta todos los pedidos de un HttpClient (front).
DI (inyección de dependencias)	Las clases piden lo que necesitan por constructor y un contenedor se lo entrega.
DTO	Clase que solo transporta datos entre la API y el cliente.
Error en cascada	Error que en realidad es consecuencia de otro anterior.
Fallback policy	Regla del back: todos los endpoints exigen autenticación salvo los marcados como AllowAnonymous.
HttpClient	Clase de .NET que manda pedidos HTTP.
JWT	Token firmado que identifica al usuario y vence; lo devuelve el login.
Lifetime	Cuánto vive una instancia registrada en DI: Singleton, Transient o Scoped.
Middleware	Pieza que intercepta todos los pedidos que llegan a la API (back).
MVVM	Model-View-ViewModel: patrón que separa la interfaz de la lógica.
Pager	Visor de texto donde git muestra salidas largas; se sale con q.
Patrón Result	Devolver un objeto con éxito/datos o error/mensaje en lugar de lanzar excepciones.
PR (Pull Request)	Pedido para integrar una rama a main, con revisión del equipo.
SecureStorage	Almacenamiento cifrado por el sistema operativo.
Service locator	Buscar servicios a mano desde el contenedor; mala práctica fuera de pruebas.
Shell	Esqueleto de navegación de MAUI (AppShell).
StaticResource	Referencia a un recurso por nombre (por ejemplo un color); se resuelve al cargar el XAML.
Target framework	Plataforma para la que se compila (net10.0-android, net10.0-windows...).
Untracked	Archivo que existe en la carpeta pero git nunca registró.
User secrets	Configuración sensible guardada fuera del proyecto, en la PC de cada dev.
Workload	Componente instalable de Visual Studio (por ejemplo, MAUI).
XAML	Lenguaje de marcado con el que se describen las pantallas de MAUI.

Apéndice A. Código final de referencia

Versión final de cada archivo de la Fase 0, para consultar sin recorrer los pasos.

NutriTrack.MAUI.csproj (fragmentos modificados)

```
<TargetFrameworks>net10.0-android</TargetFrameworks>
<TargetFrameworks
  Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net10.0-windows10.0.19041.
0</TargetFrameworks>

<!-- Nombre visible de la app (barra de la ventana / nombre bajo el ícono en Android) -->
<ApplicationTitle>NutriTrack</ApplicationTitle>

<!-- Ícono y pantalla de carga con el verde de NutriTrack -->
<MauiIcon Include="Resources\AppIcon\appicon.svg" ForegroundFile="Resources\AppIcon\appiconfg.svg"
  Color="#00A63E" />
<MauiSplashScreen Include="Resources\Splash\splash.svg" Color="#00A63E" BaseSize="128,128" />

<!-- Paquetes NuGet agregados (las versiones pueden variar) -->
<PackageReference Include="CommunityToolkit.Mvvm" Version="..." />
<PackageReference Include="Microsoft.Extensions.Http" Version="10..." />
```

MauiProgram.cs

```
using Microsoft.Extensions.Logging;
using NutriTrack.MAUI.Services;
using NutriTrack.MAUI.Views;

namespace NutriTrack.MAUI
{
    public static class MauiProgram
    {
        public static MauiApp CreateMauiApp()
        {
            var builder = MauiApp.CreateBuilder();
            builder
                .UseMauiApp<App>()
                .ConfigureFonts(fonts =>
                {
                    fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
                    fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
                });

            // Sesión del usuario: Singleton porque tiene que ser UNA sola
            // instancia compartida por toda la app
            builder.Services.AddSingleton<ISesionService, SesionService>();

            // Handler que agrega el token a cada pedido
            builder.Services.AddTransient<AuthHeaderHandler>();
        }
    }
}
```

```

        // Cliente HTTP para hablar con NutriTrack.API
        // Se configura una sola vez acá y todos los servicios lo reutilizan
        builder.Services.AddHttpClient("NutriTrackApi", client =>
        {
            client.BaseAddress = new Uri(ApiConfig.BaseUrl);
            client.Timeout = TimeSpan.FromSeconds(15);
        })
        .AddHttpMessageHandler<AuthHeaderHandler>();

        // Navegación y pantallas
        builder.Services.AddTransient<LoginPage>();
        builder.Services.AddTransient<PanelPrincipalPage>();

#if DEBUG
        builder.Logging.AddDebug();
#endif

        return builder.Build();
    }
}

```

App.xaml.cs

```

using NutriTrack.MAUI.Services;

namespace NutriTrack.MAUI
{
    public partial class App : Application
    {
        private readonly ISessionService _sessionService;

        public App(ISessionService sessionService)
        {
            InitializeComponent(); // 1) carga Colors.xaml y Styles.xaml
            UserAppTheme = AppTheme.Light; // tema claro fijo (como el prototipo)
            _sessionService = sessionService;
        }

        protected override Window CreateWindow(IActivationState? activationState)
        {
            // 2) recién acá se crea el Shell, cuando los colores ya existen
            return new Window(new AppShell(_sessionService));
        }
    }
}

```

AppShell.xaml

```
<?xml version="1.0" encoding="UTF-8" ?>
<Shell
  x:Class="NutriTrack.MAUI.AppShell"
  xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:views="clr-namespace:NutriTrack.MAUI.Views"
  Shell.FlyoutBehavior="Disabled"
  Shell.BackgroundColor="{StaticResource Primary}"
  Shell.ForegroundColor="White"
  Shell.TitleColor="White"
  Title="NutriTrack">

  <!-- Pantallas principales: se navega a ellas con "//ruta" (borra el historial) -->
  <ShellContent Route="login"
    ContentTemplate="{DataTemplate views:LoginPage}" />

  <ShellContent Route="panel"
    Title="Panel Principal"
    ContentTemplate="{DataTemplate views:PanelPrincipalPage}" />

</Shell>
```

AppShell.xaml.cs

```
using NutriTrack.MAUI.Services;

namespace NutriTrack.MAUI
{
    public partial class AppShell : Shell
    {
        private readonly ISessionService _sessionService;

        public AppShell(ISessionService sessionService)
        {
            InitializeComponent();

            // Acá se registran las pantallas que se apilan (navegación relativa), por ejemplo:
            // Routing.RegisterRoute("animales", typeof(AnimalesPage));

            _sessionService = sessionService;
            _sessionService.SesionExpirada += OnSesionExpirada;

            Loaded += OnLoaded;
        }

        // Al abrir la app: si hay una sesión guardada y vigente, ir directo al panel.
        private async void OnLoaded(object? sender, EventArgs e)
        {
            if (await _sessionService.RestaurarSesionAsync())
                await GoToAsync("//panel");
        }

        // La API rechazó el token (vencido o inválido): avisar y volver al login.
        private void OnSesionExpirada(object? sender, EventArgs e)
        {
            MainThread.BeginInvokeOnMainThread(async () =>
            {
                await DisplayAlertAsync("Sesión expirada",
                    "Tu sesión expiró. Vuelve a iniciar sesión.", "Aceptar");
                await GoToAsync("//login");
            });
        }
    }
}
```

Services/ApiConfig.cs

```
namespace NutriTrack.MAUI.Services
{
    public static class ApiConfig
    {
#if ANDROID
        // El emulador de Android llega a la PC a través de 10.0.2.2
        // ("localhost" dentro del emulador sería el propio emulador).
        public const string BaseUrl = "http://10.0.2.2:5068/";
#else
        public const string BaseUrl = "http://localhost:5068/";
#endif
    }
}
```

Services/ISesionService.cs

```
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    public interface ISesionService
    {
        // Datos del usuario logueado (null si no hay sesión).
        SesionUsuario? SesionActual { get; }

        bool HaySesionActiva { get; }

        // Se dispara cuando la API rechaza el token (vencido o inválido).
        event EventHandler? SesionExpirada;

        Task IniciarSesionAsync(SesionUsuario sesion);
        Task<bool> RestaurarSesionAsync();
        void CerrarSesion();
        void NotificarSesionExpirada();
    }
}
```

Services/SesionService.cs

```
using System.Text.Json;
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    public class SesionService : ISesionService
    {
        private const string ClaveAlmacenamiento = "sesion_nutritrack";

        public SesionUsuario? SesionActual { get; private set; }

        public bool HaySesionActiva => SesionActual is { EstaVigente: true };

        public event EventHandler? SesionExpirada;

        public async Task IniciarSesionAsync(SesionUsuario sesion)
        {
            SesionActual = sesion;

            try
            {
                var json = JsonSerializer.Serialize(sesion);
                await SecureStorage.Default.SetAsync(ClaveAlmacenamiento, json);
            }
            catch (Exception)
            {
                // Si el almacenamiento seguro falla, la sesión sigue
                // funcionando en memoria; solo no se recordará al reabrir.
            }
        }

        public async Task<bool> RestaurarSesionAsync()
        {
            try
            {
                var json = await SecureStorage.Default.GetAsync(ClaveAlmacenamiento);
                if (string.IsNullOrEmpty(json))
                    return false;

                var sesion = JsonSerializer.Deserialize<SesionUsuario>(json);
                if (sesion is null || !sesion.EstaVigente)
                {
                    CerrarSesion();
                    return false;
                }

                SesionActual = sesion;
                return true;
            }
            catch (Exception)
            {
                CerrarSesion();
            }
        }
    }
}
```

```

        return false;
    }
}

public void CerrarSesion()
{
    SesionActual = null;
    SecureStorage.Default.Remove(ClaveAlmacenamiento);
}

public void NotificarSesionExpirada()
{
    CerrarSesion();
    SesionExpirada?.Invoke(this, EventArgs.Empty);
}
}
}

```

Services/AuthHeaderHandler.cs

```

using System.Net;
using System.Net.Http.Headers;

namespace NutriTrack.MAUI.Services
{
    // Se ejecuta en TODOS los pedidos del cliente "NutriTrackApi":
    // 1) Si hay sesión, agrega el token en el encabezado Authorization.
    // 2) Si la API responde 401 teniendo sesión, avisa que la sesión expiró.
    public class AuthHeaderHandler : DelegatingHandler
    {
        private readonly ISesionService _sesionService;

        public AuthHeaderHandler(ISesionService sesionService)
        {
            _sesionService = sesionService;
        }

        protected override async Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request, CancellationToken cancellationToken)
        {
            var sesion = _sesionService.SesionActual;

            if (sesion is not null)
            {
                request.Headers.Authorization =
                    new AuthenticationHeaderValue("Bearer", sesion.Token);
            }

            // Acá el pedido sale realmente hacia la API y esperamos la respuesta.
            var respuesta = await base.SendAsync(request, cancellationToken);

            if (respuesta.StatusCode == HttpStatusCode.Unauthorized && sesion is not null)
            {
                _sesionService.NotificarSesionExpirada();
            }

            return respuesta;
        }
    }
}

```

Services/ApiServiceBase.cs

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using System.Text.Json.Serialization;
using NutriTrack.MAUI.Models;

namespace NutriTrack.MAUI.Services
{
    // Clase base para todos los servicios que hablan con la API
    // (AuthService, AnimalService, RodeoService...).
    // Centraliza el manejo de errores para que ninguna pantalla tenga que hacerlo.
    public abstract class ApiServiceBase
    {
        protected readonly HttpClient Http;

        // Misma reglas de JSON que la API: camelCase y enums como texto.
        protected static readonly JsonSerializerOptions OpcionesJson =
            new(JsonSerializerDefaults.Web)
            {
                Converters = { new JsonStringEnumConverter() }
            };

        protected ApiServiceBase(IHttpClientFactory httpClientFactory)
        {
            Http = httpClientFactory.CreateClient("NutriTrackApi");
        }

        // ----- Pedidos que devuelven datos -----

        protected Task<ResultadoApi<T>> GetAsync<T>(string url) =>
            EnviarAsync<T>(() => Http.GetAsync(url), leerDatos: true);

        protected Task<ResultadoApi<T>> PostAsync<T>(string url, object cuerpo) =>
            EnviarAsync<T>(() => Http.PostAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos: true);

        protected Task<ResultadoApi<T>> PutAsync<T>(string url, object cuerpo) =>
            EnviarAsync<T>(() => Http.PutAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos: true);

        // ----- Pedidos donde solo importa si salió bien -----

        protected Task<ResultadoApi<bool>> PostSinRespuestaAsync(string url, object cuerpo) =>
            EnviarAsync<bool>(() => Http.PostAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos:
                false);

        protected Task<ResultadoApi<bool>> PutSinRespuestaAsync(string url, object cuerpo) =>
            EnviarAsync<bool>(() => Http.PutAsJsonAsync(url, cuerpo, OpcionesJson), leerDatos:
                false);

        protected Task<ResultadoApi<bool>> DeleteAsync(string url) =>
            EnviarAsync<bool>(() => Http.DeleteAsync(url), leerDatos: false);

        // ----- El corazón: un solo lugar que maneja todos los casos -----
    }
}
```

```

private async Task<ResultadoApi<T>> EnviarAsync<T>(
    Func<Task<HttpResponseMessage>> pedido, bool leerDatos)
{
    try
    {
        using var respuesta = await pedido();

        if (!respuesta.IsSuccessStatusCode)
        {
            var mensaje = await LeerMensajeErrorAsync(respuesta);
            return ResultadoApi<T>.Error(mensaje, respuesta.StatusCode);
        }

        if (!leerDatos)
            return ResultadoApi<T>.Ok(default);

        // Algunos endpoints devuelven un texto plano en vez de JSON.
        if (typeof(T) == typeof(string))
        {
            var texto = await respuesta.Content.ReadAsStringAsync();
            return ResultadoApi<T>.Ok((T)(object)texto);
        }

        var datos = await respuesta.Content.ReadFromJsonAsync<T>(OpcionesJson);
        return ResultadoApi<T>.Ok(datos);
    }
    catch (TaskCanceledException)
    {
        return ResultadoApi<T>.Error(
            "El servidor tardó demasiado en responder. Intentá de nuevo.");
    }
    catch (HttpRequestException)
    {
        return ResultadoApi<T>.Error(
            "No se pudo conectar con el servidor. Verificá tu conexión a internet.");
    }
    catch (JsonException)
    {
        return ResultadoApi<T>.Error(
            "La respuesta del servidor no tiene el formato esperado.");
    }
}

// Convierte la respuesta de error de la API en un mensaje para el usuario.
private static async Task<string> LeerMensajeErrorAsync(HttpResponseMessage respuesta)
{
    // Errores 500: nunca mostramos el detalle interno del servidor.
    if ((int)respuesta.StatusCode >= 500)
        return MensajePorDefecto(respuesta.StatusCode);

    var contenido = (await respuesta.Content.ReadAsStringAsync()).Trim();

```

```

        if (string.IsNullOrEmpty(contenido))
            return MensajePorDefecto(respuesta.StatusCode);

        // Caso 1: errores de validación automáticos de ASP.NET (JSON con "errors").
        if (contenido.StartsWith('{'))
        {
            try
            {
                using var documento = JsonDocument.Parse(contenido);
                if (documento.RootElement.TryGetProperty("errors", out var errores))
                {
                    foreach (var campo in errores.EnumerateObject())
                        foreach (var detalle in campo.Value.EnumerateArray())
                            return detalle.GetString() ??
                                MensajePorDefecto(respuesta.StatusCode);
                }
            }
            catch (JsonException) { }

            return MensajePorDefecto(respuesta.StatusCode);
        }

        // Caso 2: mensajes de texto de los controllers, ej. BadRequest("La caravana ya
        // existe").
        return contenido.Trim('');
    }

    private static string MensajePorDefecto(HttpStatusCode codigo) => codigo switch
    {
        HttpStatusCode.BadRequest => "Los datos enviados no son válidos.",
        HttpStatusCode.Unauthorized => "Tu sesión expiró. Volvé a iniciar sesión.",
        HttpStatusCode.Forbidden => "No tenés permisos para realizar esta acción.",
        HttpStatusCode.NotFound => "No se encontró lo que buscabas.",
        _ when (int)codigo >= 500 => "Ocurrió un error en el servidor. Intentá de nuevo más
            tarde.",
        _ => "Ocurrió un error inesperado."
    };
}

```

Models/RolUsuario.cs

```

namespace NutriTrack.MAUI.Models
{
    // IMPORTANTE: los nombres tienen que coincidir EXACTAMENTE con
    // NutriTrack.Core.Entities.Enums.RolUsuario, porque la API los
    // manda como texto ("Administrador", "EncargadoDeCampo"...).
    public enum RolUsuario
    {
        AsesorTecnico,
        EncargadoDeCampo,
        Administrador
    }
}

```

Models/SesionUsuario.cs

```
namespace NutriTrack.MAUI.Models
{
    // Datos del usuario logueado. Se arman a partir de la respuesta
    // de POST api/login y se guardan mientras la sesión esté vigente.
    public class SesionUsuario
    {
        public string Token { get; set; } = string.Empty;
        public DateTime Expiracion { get; set; }
        public string NombreUsuario { get; set; } = string.Empty;
        public RolUsuario Rol { get; set; }

        public bool EstaVigente => Expiracion > DateTime.UtcNow;
    }
}
```

Models/ResultadoApi.cs

```
using System.Net;

namespace NutriTrack.MAUI.Models
{
    // Resultado de cualquier pedido a la API.
    // Si Exito es true, los datos están en Datos.
    // Si es false, MensajeError trae un texto listo para mostrar al usuario.
    public class ResultadoApi<T>
    {
        public bool Exito { get; private init; }
        public T? Datos { get; private init; }
        public string? MensajeError { get; private init; }
        public HttpStatusCode? CódigoEstado { get; private init; }

        public static ResultadoApi<T> Ok(T? datos) =>
            new() { Exito = true, Datos = datos };

        public static ResultadoApi<T> Error(string mensaje, HttpStatusCode? codigo = null) =>
            new() { Exito = false, MensajeError = mensaje, CódigoEstado = codigo };
    }
}
```

ViewModels/BaseViewModel.cs

```

using CommunityToolkit.Mvvm.ComponentModel;

namespace NutriTrack.MAUI.ViewModels
{
    // Clase base de todos los ViewModels de NutriTrack.
    // Centraliza el estado "cargando" y el mensaje de error que comparten las pantallas.
    public partial class BaseViewModel : ObservableObject
    {
        // true mientras se espera una respuesta de la API.
        [ObservableProperty]
        [NotifyPropertyChangedFor(nameof(NoEstaOcupado))]
        public partial bool EstaOcupado { get; set; }

        // Lo contrario de EstaOcupado, para habilitar botones desde el XAML.
        public bool NoEstaOcupado => !EstaOcupado;

        // Mensaje de error para mostrar en pantalla (null si no hay error).
        [ObservableProperty]
        [NotifyPropertyChangedFor(nameof(HayError))]
        public partial string? MensajeError { get; set; }

        public bool HayError => !string.IsNullOrEmpty(MensajeError);

        // Ejecuta una operación mostrando "cargando" y limpiando errores anteriores.
        // Uso en cualquier ViewModel: await EjecutarAsync(async () => { ... });
        protected async Task EjecutarAsync(Func<Task> operacion)
        {
            if (EstaOcupado)
                return;

            try
            {
                EstaOcupado = true;
                MensajeError = null;
                await operacion();
            }
            finally
            {
                EstaOcupado = false;
            }
        }
    }
}

```

Views/LoginPage.xaml

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="NutriTrack.MAUI.Views.LoginPage"
    Shell.NavBarIsVisible="False"
    BackgroundColor="{StaticResource Fondo}">

    <!-- PROVISORIA: el diseño real se hace en la Fase 1 -->
    <VerticalStackLayout Spacing="16" VerticalOptions="Center" HorizontalOptions="Center">
        <Label Text="NutriTrack"
            FontSize="32"
            FontAttributes="Bold"
            TextColor="{StaticResource TextoPrincipal}"
            HorizontalOptions="Center" />
        <Label Text="Pantalla de login (en construcción)"
            TextColor="{StaticResource TextoSecundario}"
            HorizontalOptions="Center" />
        <Button Text="Entrar (prueba de navegación)"
            Clicked="OnEntrarClickado" />
    </VerticalStackLayout>
</ContentPage>

```

Views/LoginPage.xaml.cs

```
namespace NutriTrack.MAUI.Views
{
    public partial class LoginPage : ContentPage
    {
        public LoginPage()
        {
            InitializeComponent();

            // PROVISORIO: en la Fase 1 esto lo reemplaza el LoginViewModel.
            private async void OnEntrarClicked(object? sender, EventArgs e)
            {
                await Shell.Current.GoToAsync("//panel");
            }
        }
    }
}
```

Views/PanelPrincipalPage.xaml

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="NutriTrack.MAUI.Views.PanelPrincipalPage"
    Title="Panel Principal"
    BackgroundColor="{StaticResource Fondo}">

    <!-- PROVISORIA: el diseño real se hace en la Fase 2 -->
    <VerticalStackLayout Spacing="16" VerticalOptions="Center" HorizontalOptions="Center">
        <Label Text="Panel principal (en construcción)"
            TextColor="{StaticResource TextoSecundario}"
            HorizontalOptions="Center" />
        <Button Text="Cerrar sesión (prueba de navegación)"
            Clicked="OnCerrarSesionClicked" />
    </VerticalStackLayout>
</ContentPage>
```

Views/PanelPrincipalPage.xaml.cs

```
namespace NutriTrack.MAUI.Views
{
    public partial class PanelPrincipalPage : ContentPage
    {
        public PanelPrincipalPage()
        {
            InitializeComponent();

            // PROVISORIO: en la Fase 1 esto también va a cerrar la sesión real.
            private async void OnCerrarSesionClicked(object? sender, EventArgs e)
            {
                await Shell.Current.GoToAsync("//login");
            }
        }
    }
}
```

Resources/Styles/Colors.xaml

```
<?xml version="1.0" encoding="UTF-8" ?>
<ResourceDictionary
    xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml">

    <!-- Note: For Android please see also Platforms\Android\Resources\values\colors.xml -->

    <!-- Paleta NutriTrack (tomada del prototipo de Figma) -->
    <Color x:Key="Primary">#00A63E</Color>
    <Color x:Key="PrimaryDark">#008236</Color>
    <Color x:Key="PrimaryDarkText">#242424</Color>
    <Color x:Key="Secondary">#F0FDF4</Color>
    <Color x:Key="SecondaryDarkText">#008236</Color>
    <Color x:Key="Tertiary">#016630</Color>

    <!-- Colores propios de NutriTrack -->
    <Color x:Key="Fondo">#F9FAFB</Color>
    <Color x:Key="Superficie">#FFFFFF</Color>
    <Color x:Key="Borde">#E5E7EB</Color>
    <Color x:Key="TextoPrincipal">#101828</Color>
    <Color x:Key="TextoSecundario">#6A7282</Color>
    <Color x:Key="Peligro">#E7000B</Color>
    <Color x:Key="PeligroSuave">#FEF2F2</Color>

    <!-- Colores base que usa Styles.xaml (NO BORRAR) -->
    <Color x:Key="White">White</Color>

    <Color x:Key="Black">Black</Color>
    <Color x:Key="Magenta">#D600AA</Color>
    <Color x:Key="MidnightBlue">#190649</Color>
    <Color x:Key="OffBlack">#1f1f1f</Color>

    <Color x:Key="Gray100">#E1E1E1</Color>
    <Color x:Key="Gray200">#C8C8C8</Color>
    <Color x:Key="Gray300">#ACACAC</Color>
    <Color x:Key="Gray400">#919191</Color>
    <Color x:Key="Gray500">#6E6E6E</Color>
    <Color x:Key="Gray600">#404040</Color>
    <Color x:Key="Gray900">#212121</Color>
    <Color x:Key="Gray950">#141414</Color>

    <SolidColorBrush x:Key="PrimaryBrush" Color="{StaticResource Primary}"/>
    <SolidColorBrush x:Key="SecondaryBrush" Color="{StaticResource Secondary}"/>
    <SolidColorBrush x:Key="TertiaryBrush" Color="{StaticResource Tertiary}"/>
    <SolidColorBrush x:Key="WhiteBrush" Color="{StaticResource White}"/>
    <SolidColorBrush x:Key="BlackBrush" Color="{StaticResource Black}"/>

    <SolidColorBrush x:Key="Gray100Brush" Color="{StaticResource Gray100}"/>
    <SolidColorBrush x:Key="Gray200Brush" Color="{StaticResource Gray200}"/>
    <SolidColorBrush x:Key="Gray300Brush" Color="{StaticResource Gray300}"/>
    <SolidColorBrush x:Key="Gray400Brush" Color="{StaticResource Gray400}"/>
    <SolidColorBrush x:Key="Gray500Brush" Color="{StaticResource Gray500}"/>
    <SolidColorBrush x:Key="Gray600Brush" Color="{StaticResource Gray600}"/>

    <SolidColorBrush x:Key="Gray900Brush" Color="{StaticResource Gray900}"/>
    <SolidColorBrush x:Key="Gray950Brush" Color="{StaticResource Gray950}"/>
</ResourceDictionary>
```

Platforms/Android/Resources/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="colorPrimary">#00A63E</color>
    <color name="colorPrimaryDark">#008236</color>
    <color name="colorAccent">#008236</color>
</resources>
```


Apéndice B. Descripción del PR de la Fase 0

Título: chore(maui): estructura base del front (Fase 0). Base: main ← chore/maui-estructura-base.

Markdown para GitHub

Qué se hizo

Base del front en MAUI para que cada uno pueda armar sus pantallas encima con la misma estructura.

- Se dejaron solo los targets **Windows** y **Android** (alcance del proyecto) y se borraron las carpetas de iOS/Mac.
- Paquetes: `CommunityToolkit.Mvvm` (MVVM)` y `Microsoft.Extensions.Http` (HttpClient)`.
- Paleta de colores del prototipo en `Colors.xaml`` y **tema claro fijo**.
- Conexión con la API: `ApiConfig` (URL por plataforma)` + `HttpClient`` registrado como `"NutriTrackApi"`.
- **Sesión**: `SesionService`` guarda el token cifrado (`SecureStorage``) y permite recordar la sesión.
- **Token automático**: `AuthHeaderHandler`` agrega `Authorization: Bearer`` a cada pedido. Si la API responde 401 con sesión activa, se cierra la sesión y se vuelve al login.
- **Manejo de errores**: `ApiServiceBase`` + `ResultadoApi<T>``.
- `BaseViewModel`` con `EstaOcupado``, `MensajeError`` y `EjecutarAsync`` (evita doble envío).
- Navegación con Shell: pantallas **provisorias** de Login y Panel Principal. Se quitó la plantilla.

Cómo agregar una pantalla nueva (convención)

1. Service: heredar de `ApiServiceBase`` (`GetAsync<T>``, `PostAsync<T>`...`).
2. ViewModel: heredar de `BaseViewModel`` y envolver las llamadas en `EjecutarAsync(...)``.
3. View: `ContentPage (XAML)`` en `Views/``, con Binding al ViewModel.
4. Registrar en `MauiParam.cs``.
5. Rutas que se apilan: `Routing.RegisterRoute("animales", typeof(AnimalesPage));`` en `AppShell.xaml.cs``.

`"/login`` y `"/panel`` son absolutas: solo para entrar y salir de la sesión.
6. Colores: usar las claves de `Colors.xaml``, no valores hex sueltos.

Cómo probar

- API con el perfil http: `dotnet run --project NutriTrack.API --launch-profile http``
- MAUI con Windows Machine.
- Login provisorio → "Entrar" → Panel (barra verde, sin flecha de volver) → "Cerrar sesión" → Login.

Pendiente de probar (depende del login real, Fase 1)

- Guardar y restaurar la sesión al reabrir la app.
- Token en los pedidos y vuelta al login ante un 401.
- Mensajes de error de `ApiServiceBase`` contra respuestas reales.

Lo que sí se probó: compilación sin errores, conexión con la API (GET api/Ingrediente → 200) y la navegación Login ↔ Panel.

Ojo

Si `main`` todavía no compila la API por el `using`` faltante en `UsuarioResponseDTO.cs`` (lo arregla `fix/using-rolusuario-usuario-response``), no es por este PR: solo toca `NutriTrack.MAUI``.

Capturas

(login y panel provisorios)